

# La Federación Galáctica de Recursos

## Simulación Concurrente y Distribuida

---

## Índice

### 1. Análisis de alto nivel

- 1.1 Descripción general del problema*
- 1.2 Identificación de los actores principales*
- 1.3 Flujo general del sistema*
- 1.4 Extensión distribuida y visión global servidor-cliente*

### 2. Diseño general del sistema

- 2.1 Arquitectura global*
- 2.2 Organización en paquetes*
- 2.3 Herramientas de sincronización y comunicación concurrente*
- 2.4 Módulo distribuido: comunicación remota*

### 3. Descripción de las clases principales

- 3.1 Paquete Modelo - recursos, planetas, depósitos, zonas y entidades del dominio*
- 3.2 Paquete Concurrencia - hilos de delegados, patrullas, saqueadores y lógica de refuerzos*
- 3.3 Paquete GUI - interfaz gráfica del servidor*
- 3.4 Paquete Log - sistema de registro en federacion\_galactica.txt*
- 3.5 Paquete Remoto - servidor remoto y protocolo de comunicación*
- 3.6 Paquete InterfazCliente - cliente remoto con su interfaz gráfica*
- 3.7 Paquete Main - clase de lanzamiento y configuración inicial*

### 4. Diagrama de clases

- 4.1 Vista general del sistema*
- 4.2 Detalle del núcleo concurrente (Modelo + Concurrencia + Log)*
- 4.3 Detalle del módulo distribuido (Remoto + InterfazCliente)*

### 5. Interpretaciones y consideraciones de diseño

- 5.1 Decisiones de modelado orientado a objetos*
- 5.2 Decisiones de sincronización y concurrencia*
- 5.3 Decisiones sobre el módulo distribuido*
- 5.4 Limitaciones, problemas encontrados y posibles mejoras*

Para visualización completa del proyecto, consultar también el siguiente repositorio:

<https://github.com/David17es/LaFederacionGalacticaDeRecursos-Java>

# 1. Análisis de alto nivel

## 1.1 Descripción general del problema

La Federación Galáctica de Recursos es una simulación desarrollada en Java que modela un sistema de explotación y gestión de recursos naturales a escala intergaláctica. El contexto narrativo parte de un escenario postbélico en el que la Federación Unida de Planetas trata de restablecer el comercio entre cinco planetas mineros y las diez razas que conviven en la alianza. El objetivo principal de la práctica es trasladar ese escenario a un programa concurrente y distribuido, donde múltiples actores actúan en paralelo, compiten por recursos compartidos y deben coordinarse correctamente para evitar inconsistencias en el estado del sistema.

La simulación se organiza en dos grandes bloques. El primero cubre la programación concurrente: distintos hilos representan a delegados comerciales, patrullas federales y saqueadores espaciales, todos ellos accediendo simultáneamente a zonas con capacidad limitada. El segundo bloque añade un módulo de programación distribuida basado en RMI, que permite a un cliente externo consultar el estado del sistema en tiempo real y vaciar depósitos de forma remota, sin que el usuario tenga que interactuar para actualizar la información.

## 1.2 Identificación de los actores principales

El sistema cuenta con tres tipos de actores activos, modelados obligatoriamente como hilos de ejecución, más una serie de zonas y recursos que actúan como estado compartido:

- **Delegados comerciales (DXXX):** representan a las razas de la alianza. Se generan de uno en uno cada 2 segundos hasta un máximo de 20 activos, con la posibilidad de crear nuevos si algún depósito baja del 10% de su capacidad. Su ciclo de trabajo consiste en preparar una solicitud en el Centro de Coordinación Federal, dirigirse a un planeta a extraer recursos y depositar lo obtenido en el depósito orbital correspondiente.
- **Patrullas federales (PXXX):** defienden planetas y depósitos frente a los ataques de saqueadores. Parten del Hangar de Patrullas y patrullan aleatoriamente por las distintas zonas del sistema. Pueden ser derrotadas en combate y enviadas a la Zona de Recuperación. Se generan nuevas patrullas de forma automática cuando el sistema acumula recursos suficientes (150 de cristal, 100 de mineral y 75 de plasma), hasta un máximo de 20 simultáneas.
- **Saqueadores espaciales (SXXX):** intentan atacar planetas y depósitos. Aparecen automáticamente cada 10-20 segundos en la Base de Saqueadores, con un máximo de 40 simultáneos. Escogen objetivos con una probabilidad del 70% hacia depósitos y del 30% hacia planetas. Si ganan el combate contra una patrulla, saquean el recurso almacenado o interrumpen la extracción en el planeta.

Las zonas del sistema son: el Centro de Coordinación Federal, cinco planetas mineros (Cryon, Velora, Ferrum, Drax e Ignis), tres depósitos orbitales (Cristal, Mineral y Plasma), el Hangar de Patrullas, la Base de Saqueadores y la Zona de Recuperación.

## *1.3 Flujo general del sistema*

Cuando arranca la simulación, se lanzan los generadores de delegados, patrullas y saqueadores como hilos independientes. Los delegados comerciales siguen un ciclo continuo: esperan en el Centro de Coordinación Federal, seleccionan un recurso de forma aleatoria, acceden a uno de los planetas que lo producen (con un máximo de 4 delegados por planeta), extraen entre 10 y 25 unidades, y depositan esa cantidad en el depósito correspondiente (máximo 3 delegados depositando a la vez). Si un planeta es atacado mientras un delegado está extrayendo, este es expulsado sin obtener recursos y se recupera en la Zona de Recuperación entre 10 y 15 segundos.

Las patrullas circulan entre zonas de forma continua. Si una patrulla se encuentra en una zona en el momento en que un saqueador la ataca, se produce un combate con un 50% de probabilidad de victoria para cada parte. El saqueador que pierde vuelve a su base y espera 20 segundos; la patrulla derrotada va a la Zona de Recuperación. Si no hay patrullas en la zona atacada, el saqueador actúa sin resistencia y procede directamente al saqueo.

El mecanismo de refuerzos automáticos añade dinamismo al sistema: cada vez que los depósitos acumulan el umbral necesario, se consumen esos recursos y se crea una nueva patrulla. Este proceso debe protegerse adecuadamente, ya que pueden coincidir depósitos de delegados, saqueos de piratas y comprobaciones de refuerzo en el mismo instante.

## *1.4 Extensión distribuida*

La segunda parte del sistema incorpora un módulo remoto basado en RMI (Remote Method Invocation). El servidor expone una interfaz remota (InterfazMonitorRemoto) que el cliente puede invocar para consultar en tiempo real el número de delegados y patrullas en cada zona, el estado de los recursos almacenados, el número de saqueadores activos y la situación de la Zona de Recuperación. Además, el cliente permite vaciar de forma independiente cada uno de los tres depósitos orbitales. Toda esta información se actualiza automáticamente sin que el usuario deba pulsar ningún botón, lo que requiere que el cliente sondee el servidor de forma periódica o que el servidor notifique los cambios.

# **2. Diseño general del sistema**

## *2.1 Arquitectura global*

El sistema se divide en dos programas diferenciados: el servidor (la simulación principal con su interfaz gráfica local) y el cliente remoto (una aplicación independiente que se conecta al servidor mediante RMI para monitorizar y controlar el sistema a distancia). El servidor arranca la simulación, gestiona todos los hilos activos y expone el objeto remoto en el registro RMI. El cliente se conecta a ese registro, obtiene la referencia remota y comienza a actualizar su interfaz periódicamente con los datos que recibe.

Desde el punto de vista de capas, el servidor se estructura en tres niveles bien diferenciados: el modelo de dominio (estado de la galaxia, zonas, actores y recursos), la lógica concurrente (hilos y clases de sincronización) y la capa de presentación (interfaz gráfica Swing y módulo remoto). Esta separación facilita la comprensión del código y hace que cada paquete tenga una responsabilidad clara.

## 2.2 Organización de paquetes

El proyecto se organiza en los siguientes paquetes:

| Paquete         | Responsabilidad                                                              |
|-----------------|------------------------------------------------------------------------------|
| Modelo          | Clases de dominio: actores, zonas, depósitos, planetas y estado global       |
| Concurrencia    | Hilos activos (delegados, patrullas, saqueadores) y clases de sincronización |
| GUI             | Interfaz gráfica del servidor: ventana principal y paneles de estado         |
| Log             | Registro centralizado de eventos en federacion_galactica.txt                 |
| Remoto          | Interfaz RMI, implementación del objeto remoto, DTOs y servidor RMI          |
| InterfazCliente | Cliente remoto con su propia ventana Swing                                   |
| Main            | Punto de entrada de la aplicación y configuración inicial                    |

## 2.3 Herramientas de sincronización y comunicación concurrente

El acceso concurrente a las zonas con aforo limitado (planetas con máximo 4 delegados, depósitos con máximo 3, zonas de patrulla con máximo 3) se controla mediante Semaphore de Java. Los semáforos permiten limitar de forma sencilla y eficiente el número de hilos que acceden simultáneamente a un recurso sin necesidad de implementar lógica de conteo manual.

Para la gestión del estado compartido de los depósitos orbitales (donde pueden coincidir depósitos de delegados, saqueos de piratas y comprobaciones de refuerzo) se utilizan ReentrantLock junto con Condition. Esta combinación permite bloquear la espera cuando el depósito está lleno y despertar a los hilos correspondientes en cuanto haya espacio disponible, de manera ordenada y sin busy-waiting. Las clases SincronizacionRecursos, SincronizacionSaqueadores y SincronizacionZonas encapsulan estos mecanismos para separar la lógica de sincronización de la lógica de negocio de cada hilo.

El sistema de log (SistemaLog, en el paquete Log) también es un recurso compartido al que acceden todos los hilos simultáneamente. Para proteger la escritura en fichero se utiliza igualmente un ReentrantLock, garantizando que las líneas se escriben de forma atómica y en orden. La clase sigue el patrón Singleton, de modo que existe una única instancia en todo el sistema.

Los ataques de saqueadores introducen un caso especial de sincronización: cuando un saqueador empieza a atacar una zona, esta queda bloqueada para nuevas entradas de delegados, y las patrullas presentes participan en la defensa. Para coordinar esta situación se emplean flags de estado protegidos por locks, que notifican a los hilos en espera una vez que el ataque ha finalizado.

## 2.4 Módulo distribuido: comunicación por RMI

Para la parte distribuida se ha optado por Java RMI frente a Sockets, ya que RMI permite invocar métodos remotos de forma transparente y tipada, reduciendo la cantidad de código de protocolo necesario. La interfaz `InterfazMonitorRemoto` extiende `java.rmi.Remote` y declara todos los métodos de consulta y control que el cliente puede invocar. La clase `MonitorRemotoImpl` implementa esa interfaz extendiendo `UnicastRemoteObject`, y obtiene los datos directamente de `EstadoGalaxia`, que centraliza el estado global del servidor.

Los datos que se transfieren entre servidor y cliente se empaquetan en dos objetos serializables: `EstadoRecursosDTO`, que contiene el estado de los depósitos orbitales, y `EstadoZonasDTO`, que recoge el número de actores en cada zona. Este uso del patrón DTO (Data Transfer Object) evita que el cliente tenga acceso directo a los objetos de dominio del servidor y simplifica la serialización de los datos.

El cliente remoto (`ClienteRemotoControlador`) arranca un hilo que invoca periódicamente los métodos remotos y actualiza la interfaz gráfica (`VentanaClienteRemoto` y `PanelEstadoRemoto`) de forma automática. La botonera de vaciado de depósitos invoca directamente los métodos correspondientes del objeto remoto, ejecutando la operación en el servidor de forma síncrona.

## 3. Descripción de las clases principales

### 3.1 Paquete Modelo

El paquete `Modelo` contiene todas las clases de dominio del sistema: las entidades que representan actores, zonas y el estado global. No contienen lógica concurrente por sí mismas; su función es almacenar y exponer el estado del sistema de forma ordenada.

#### Clase Zona

| Elemento              | Descripción                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Clase base abstracta que modela cualquier zona del sistema galáctico.                                                                    |
| Atributos principales | <code>nombre</code> (String), <code>capacidadMaxima</code> (int), <code>ocupacionActual</code> (int), <code>bajoAtaque</code> (boolean). |

|                     |                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Métodos principales | Getters y setters de todos los atributos; <code>estaLlena()</code> , que comprueba si la ocupación ha alcanzado la capacidad máxima. |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------|

De Zona heredan directamente `PlanetaMinero`, `DepositoOrbital`, `CentroCoordinacionFederal`, `HangarPatrullas`, `BaseSaqueadores` y `ZonaRecuperacion`. Las últimas cuatro son subclases simples que únicamente llaman al constructor de Zona con su nombre y capacidad correspondiente.

## Clase PlanetaMinero

| Elemento              | Descripción                                                                                                                               |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Representa uno de los cinco planetas mineros de la alianza.                                                                               |
| Atributos principales | Los heredados de Zona más <code>tipoRecurso</code> ( <code>TipoRecurso</code> ), que indica qué tipo de recurso se extrae en ese planeta. |
| Métodos principales   | <code>getTipoRecurso()</code> y los heredados de Zona.                                                                                    |

## Clase DepositoOrbital

| Elemento              | Descripción                                                                                                                                                                                                                             |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Modela uno de los tres depósitos orbitales donde los delegados almacenan los recursos extraídos.                                                                                                                                        |
| Atributos principales | <code>tipoRecurso</code> ( <code>TipoRecurso</code> ), <code>capacidadMaximaUnidades</code> ( <code>int</code> ), <code>unidadesActuales</code> ( <code>int</code> ). La capacidad de acceso simultáneo (3 delegados) se hereda de Zona |
| Métodos principales   | <code>getEspacioDisponible()</code> , <code>tieneEspacio(int cantidad)</code> y <code>estaPorDebajoUmbral()</code> , que devuelve true cuando las unidades caen por debajo del 10% de la capacidad máxima.                              |

## Clase Actor

| Elemento              | Descripción                                                                                                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Clase base abstracta para todos los actores activos del sistema.                                                                                                                           |
| Atributos principales | <code>id</code> ( <code>String</code> ), <code>zonaActual</code> ( <code>Zona</code> ), <code>estado</code> ( <code>String</code> ), <code>enRecuperacion</code> ( <code>boolean</code> ). |
| Métodos principales   | Getters y setters de todos los atributos; <code>toString()</code> que devuelve una representación legible del actor con su zona actual.                                                    |

De Actor heredan `DelegadoComercial`, `PatrullaFederal` y `Saqueador`, que añaden cada una sus propios atributos de dominio (tipo de recurso seleccionado, unidades extraídas, número de ataques, etc.) y los getters y setters correspondientes. Estas clases son puramente de datos: la lógica de comportamiento reside en los hilos del paquete `Concurrencia`.

## Clase EstadoGalaxia

| Elemento              | Descripción                                                                                                                                                                                                                                               |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Contenedor global del estado de la simulación. Centraliza todas las zonas, actores activos y generadores de identificadores únicos.                                                                                                                       |
| Atributos principales | Referencias a las diez zonas del sistema (cinco planetas, tres depósitos, hangar, base, zona de recuperación y centro de coordinación), listas sincronizadas de delegados, patrullas y saqueadores activos, y tres AtomicInteger para generar IDs únicos. |
| Métodos principales   | getInstancia() (Singleton), métodos generarIdDelegado/Patrulla/Saqueador(), getters de todas las zonas, y métodos sincronizados para agregar, eliminar y consultar actores (agregarDelegado(), getNumeroDelegadosActivos(), etc.).                        |

## Enumerado TipoRecurso

Define los tres tipos de recursos del sistema: CRISTAL, MINERAL y PLASMA. Se usa en planetas, depósitos, delegados y en las clases de sincronización para seleccionar el depósito correcto en cada operación.

## 3.2 Paquete Concurrency

Este paquete contiene los hilos activos de la simulación y las tres clases de sincronización que protegen el acceso a los recursos compartidos.

## Clase HiloDelegado

| Elemento              | Descripción                                                                                                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Implementa el ciclo de vida completo de un delegado comercial como hilo de ejecución.                                                                                                      |
| Atributos principales | Referencia al DelegadoComercial que representa, referencias a EstadoGalaxia, SincronizacionZonas, SincronizacionRecursos y LoggerFederacion.                                               |
| Métodos principales   | run() con el ciclo indefinido: espera en el Centro de Coordinación, selecciona planeta, extrae recurso, deposita en el depósito correspondiente y gestiona la expulsión en caso de ataque. |

## Clase HiloPatrulla

| Elemento              | Descripción                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Implementa el comportamiento de una patrulla federal: patrullaje continuo entre zonas y participación en combates contra saqueadores. |
| Atributos principales | Referencia a PatrullaFederal, referencias a EstadoGalaxia, SincronizacionZonas, SincronizacionSaqueadores y LoggerFederacion.         |

|                     |                                                                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Métodos principales | run() con el ciclo de patrullaje; lógica de combate con resultado aleatorio al 50%; gestión de la Zona de Recuperación si la patrulla es derrotada. |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

## Clase HiloSaqueador

| Elemento              | Descripción                                                                                                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Modela el comportamiento de un saqueador espacial: selección de objetivo, ataque, combate con patrullas y saqueo de recursos o expulsión de delegados.                       |
| Atributos principales | Referencia a Saqueador, referencias a EstadoGalaxia, SincronizacionZonas, SincronizacionSaqueadores, SincronizacionRecursos y LoggerFederacion.                              |
| Métodos principales   | run() con el ciclo de ataque; selección ponderada de objetivo (70% depósito/30% planeta); lógica de combate y saqueo o interrupción de extracción según el tipo de objetivo. |

## Clases GeneradorDelegados, GeneradorPatrullas, GeneradorSaqueadores

Estas tres clases son hilos cuya única responsabilidad es generar periódicamente nuevos actores y lanzar sus hilos correspondientes. GeneradorDelegados respeta el límite de 20 activos y la condición de umbral de depósito. GeneradorPatrullas controla el límite de 20 patrullas y el mecanismo de refuerzos automáticos comprobando los umbrales de recursos. GeneradorSaqueadores crea un nuevo saqueador cada 10-20 segundos con un máximo de 40 simultáneos.

## Clase ControlSimulacion

| Elemento              | Descripción                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Proporciona un mecanismo centralizado de pausa y reanudación de la simulación, utilizado tanto por la GUI local como por el cliente remoto.                                 |
| Atributos principales | pausada (volatile boolean), lock (ReentrantLock), condicion (Condition).                                                                                                    |
| Métodos principales   | pausar(), reanudar(), esperarSiPausada() - los hilos llaman a este último al inicio de cada iteración de su ciclo para quedar bloqueados cuando la simulación está pausada. |

## Clase SincronizacionZonas

| Elemento              | Descripción                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Controla el acceso por capacidad a planetas y depósitos mediante semáforos FIFO.                                    |
| Atributos principales | semaforosCapacidad: mapa de Zona a Semaphore justo (fair=true), con un semáforo por cada zona con límite de acceso. |
| Métodos principales   | entrarZona(Zona), salirZona(Zona), getHuecosDisponibles(Zona).                                                      |



## Clase SincronizacionRecursos

| Elemento              | Descripción                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Gestiona de forma thread-safe las operaciones sobre las unidades almacenadas en los depósitos.                                                                                      |
| Atributos principales | lock (ReentrantLock), hayEspacio (Condition), hayRecursos (Condition).                                                                                                              |
| Métodos principales   | depositar(TipoRecurso, int) - espera si no hay espacio; vaciarPorSaqueo(TipoRecurso) - espera si el depósito está vacío; getUnidadesActuales(TipoRecurso), hayDepositoBajoUmbral(). |

## Clase SincronizacionSaqueadores

| Elemento              | Descripción                                                                                                                                                              |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Garantiza que como máximo un saqueador ataca una misma zona en cada instante.                                                                                            |
| Atributos principales | zonasOcupadas: mapa de Zona a boolean; lock (ReentrantLock), zonaLibre (Condition).                                                                                      |
| Métodos principales   | iniciarAtaque(Zona) - bloquea si la zona ya está siendo atacada; finalizarAtaque(Zona) - libera la zona y notifica a los saqueadores en espera; estasiendoAtacada(Zona). |

## 3.3 Paquete GUI

El paquete GUI agrupa todas las clases de la interfaz gráfica del servidor, implementadas con Swing. La actualización de la GUI se realiza siempre desde el Event Dispatch Thread mediante `SwingUtilities.invokeLater()` para respetar el modelo de concurrencia de Swing.

### Clase VentanaPrincipalServidor

Ventana principal de la aplicación servidor. Organiza los distintos paneles de la interfaz (`PanelMapaGalaxia`, `PanelRecursos`, `PanelActores`, `PanelLog`) y proporciona el método `agregarLineaLog(String)` que el logger usa para mostrar eventos en pantalla en tiempo real.

### Clase PanelMapaGalaxia

Muestra visualmente el mapa de la galaxia con el estado actual de cada zona: ocupación, si está bajo ataque y los actores presentes. Se refresca periódicamente consultando `EstadoGalaxia`.

### Clase PanelRecursos

Muestra el nivel de recursos almacenados en los tres depósitos orbitales mediante barras de progreso o contadores numéricos, actualizándose en tiempo real.

## Clase PanelActores

Presenta listas actualizadas de delegados, patrullas y saqueadores, indicando el estado y la zona actual de cada uno.

## Clase PanelLog

Muestra las últimas líneas del log de eventos en un área de texto con desplazamiento automático, recibiendo las actualizaciones directamente desde `LoggerFederacionImpl`.

## 3.4 Paquete Log

### Interfaz LoggerFederacion

Define el contrato del sistema de log: un único método `registrarEvento(String descripcion)` que cualquier hilo puede invocar para registrar un evento.

### Clase LoggerFederacionImpl

| Elemento              | Descripción                                                                                                                                                                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Implementación thread-safe del logger. Escribe cada evento en el fichero <code>federacion_galactica.txt</code> y actualiza el panel de log de la GUI.                                                                                                                                 |
| Atributos principales | lock ( <code>ReentrantLock</code> ), referencia opcional a <code>VentanaPrincipalServidor</code> .                                                                                                                                                                                    |
| Métodos principales   | <code>registrarEvento(String)</code> - formatea el evento con fecha y hora, escribe en fichero bajo lock y notifica a la GUI vía <code>SwingUtilities.invokeLater()</code> ; <code>setVentana(VentanaPrincipalServidor)</code> - inyecta la referencia a la ventana tras su creación. |

El fichero se abre y se cierra en cada escritura (modo `append`), lo que garantiza que los eventos queden persistidos aunque el programa se cierre de forma inesperada. La clase `EventoLog` encapsula el mensaje con su marca de tiempo y genera la línea formateada.

## 3.5 Paquete Remoto

### Interfaz InterfazMonitorRemoto

Define los servicios RMI que el servidor expone al cliente remoto: `getEstadoRecursos()`, `getEstadoZonas()`, `pausarSimulacion()`, `reanudarSimulacion()` e `isPausada()`. Al extender `java.rmi.Remote`, todos sus métodos declaran `RemoteException`.

### Clase MonitorRemotoImpl

| Elemento | Descripción |
|----------|-------------|
|----------|-------------|

|                       |                                                                                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Implementa InterfazMonitorRemoto extendiendo UnicastRemoteObject. Consulta EstadoGalaxia y ControlSimulacion para construir las respuestas. |
| Atributos principales | Referencias a EstadoGalaxia, SincronizacionRecursos y ControlSimulacion.                                                                    |
| Métodos principales   | Implementación de todos los métodos de la interfaz remota; construcción y devolución de EstadoRecursosDTO y EstadoZonasDTO.                 |

## Clases EstadoRecursosDTO y EstadoZonasDTO

Objetos de transferencia de datos (DTO) serializables que empaquetan la información del estado de los depósitos y de los contadores de actores por zona, respectivamente. Su uso evita exponer directamente los objetos de dominio a través de la red.

## Clase ServidorRMI

Se encarga de registrar el objeto remoto en el registro RMI (LocateRegistry) con el nombre acordado, de modo que el cliente pueda localizarlo mediante una búsqueda por nombre al conectarse.

## 3.6 Paquete InterfazCliente

### Clase ClienteRemotoControlador

| Elemento              | Descripción                                                                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad       | Gestiona la conexión RMI con el servidor y lanza el hilo de actualización periódica de la interfaz del cliente.                                                                          |
| Atributos principales | Referencia a InterfazMonitorRemoto (el stub remoto), referencia a VentanaClienteRemoto.                                                                                                  |
| Métodos principales   | conectar() - localiza el objeto remoto en el registro RMI; iniciarActualizacion() - arranca un hilo que invoca periódicamente getEstadoRecursos() y getEstadoZonas() y actualiza la GUI. |

### Clase VentanaClienteRemoto

Ventana Swing del cliente remoto. Organiza el PanelEstadoRemoto y los botones de control (pausa/reanudación y vaciado de depósitos), cuyas acciones invocan directamente los métodos correspondientes del objeto remoto.

### Clase PanelEstadoRemoto

Panel que muestra en tiempo real toda la información recibida del servidor: contadores de actores por zona, nivel de recursos en los depósitos y estado de los saqueadores.

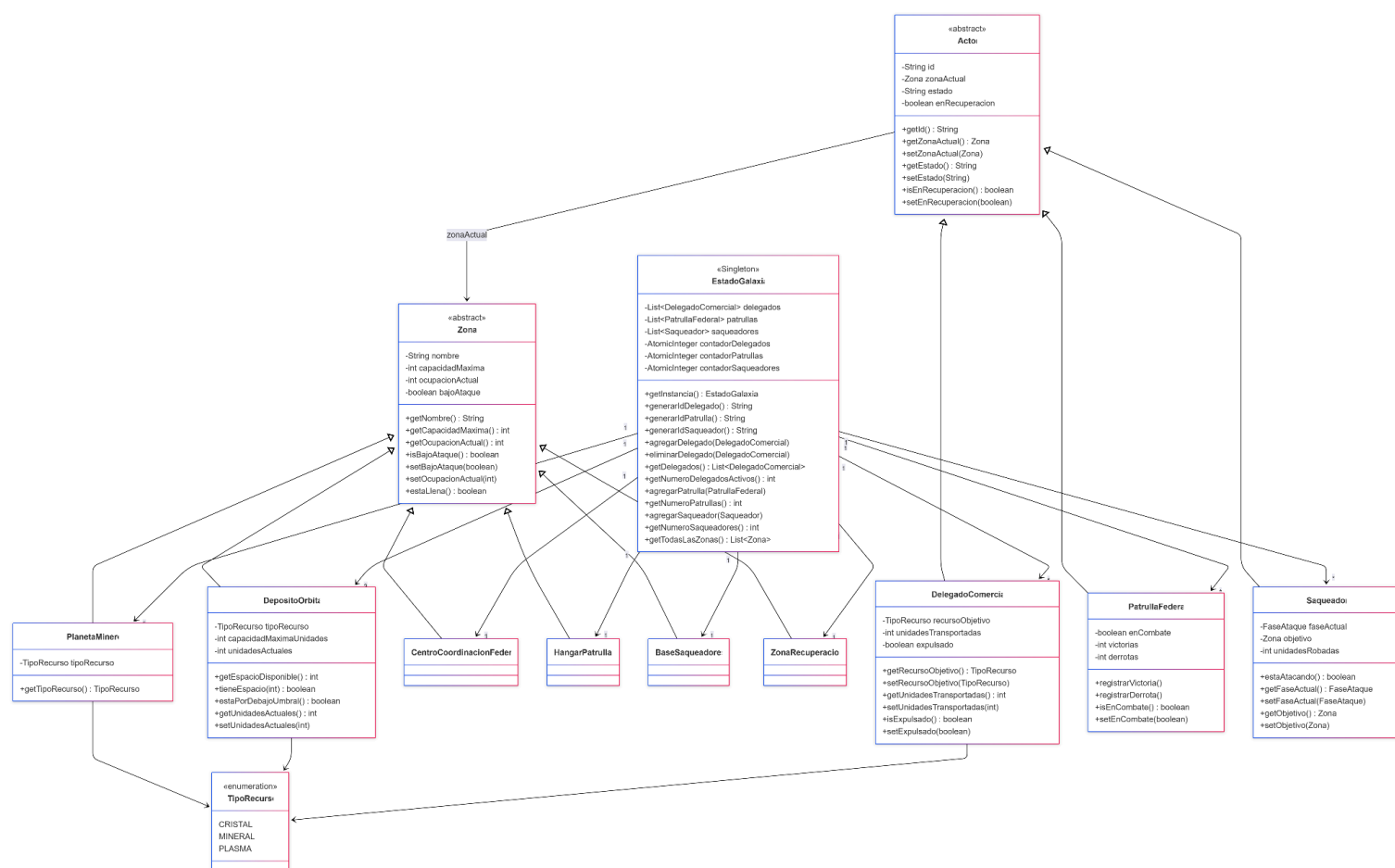
### 3.7 Paquete Main

El paquete contiene dos clases de arranque independientes:

- **MainServidor:** inicializa EstadoGalaxia, crea las instancias de las clases de sincronización, lanza los generadores de actores, arranca la GUI (VentanaPrincipalServidor) y registra el objeto remoto en el registro RMI.
- **MainClienteRemoto:** crea ClienteRemotoControlador, intenta la conexión con el servidor y, si tiene éxito, muestra VentanaClienteRemoto e inicia el ciclo de actualización automática.

## 4. Diagrama de clases

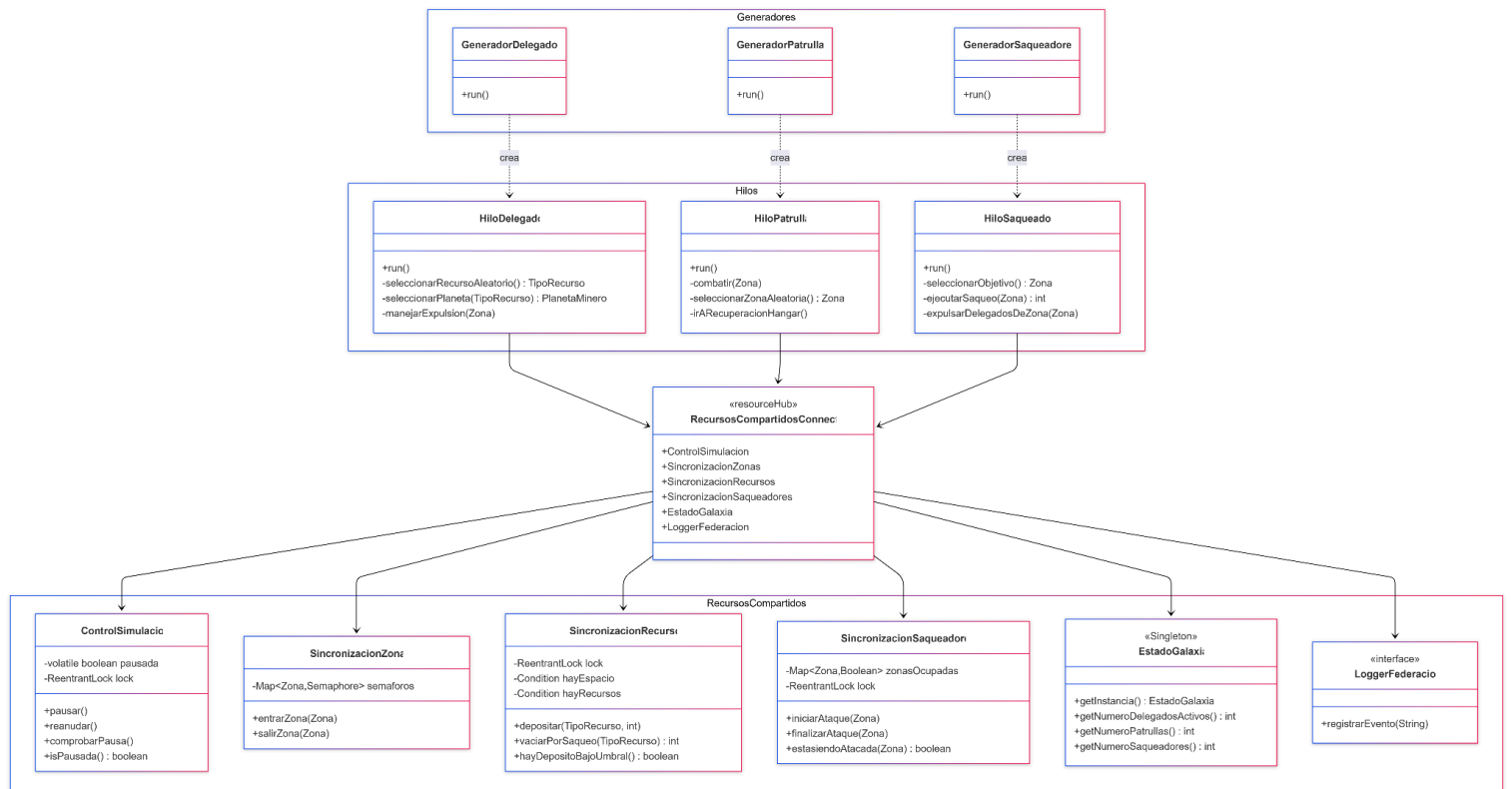
### 4.1 Vista general del sistema



La Figura 1 muestra la estructura general del modelo del dominio. En ella se representa, por un lado, la jerarquía de zonas del sistema, que engloba planetas mineros, depósitos orbitales y zonas especiales como el centro de coordinación, la base de saqueadores, el hangar de patrullas y la zona de recuperación. Por otro lado, se representa la jerarquía de actores, donde la clase abstracta Actor sirve de base para DelegadoComercial, PatrullaFederal y Saqueador. Finalmente, se incluye

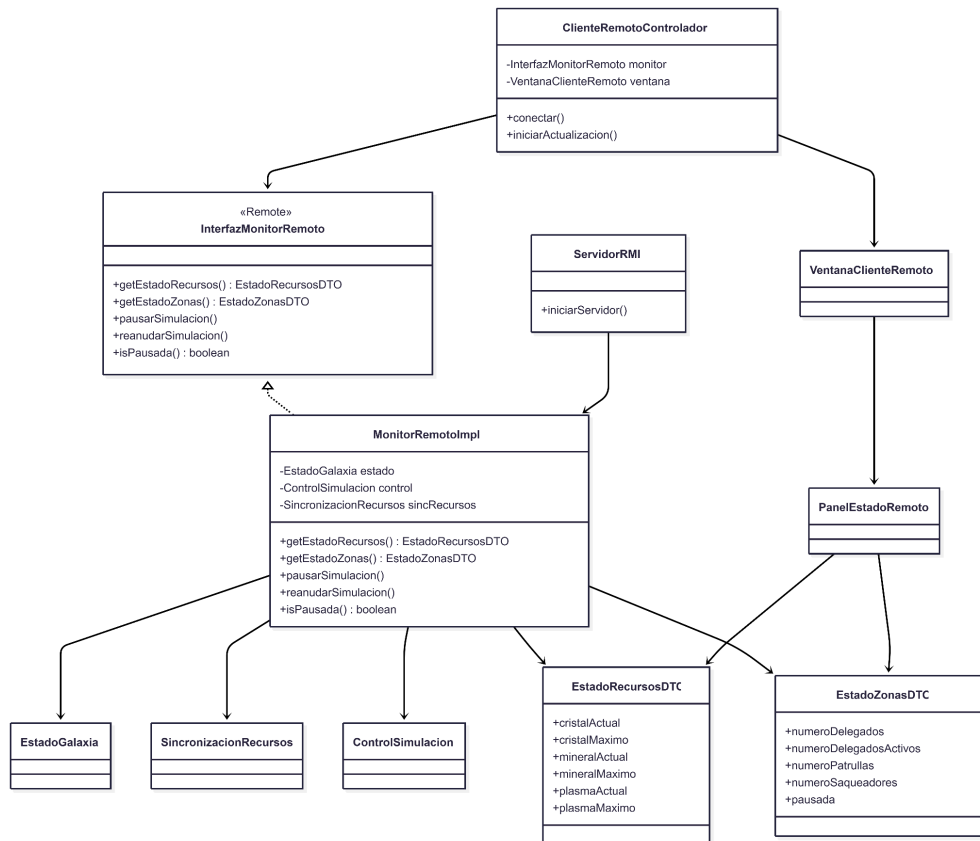
EstadoGalaxia, que actúa como contenedor global del estado y centraliza tanto las zonas como los actores activos y la generación de identificadores únicos.

## 4.2 Detalle del núcleo concurrente



La Figura 2 representa el núcleo concurrente del sistema. En ella puede observarse que los distintos tipos de hilos activos no trabajan directamente entre sí, sino a través de recursos compartidos y clases de sincronización especializadas. SincronizacionZonas controla el acceso a planetas y depósitos mediante semáforos; SincronizacionRecursos protege las operaciones de depósito y saqueo mediante locks y condiciones; y SincronizacionSaqueadores garantiza que una misma zona no pueda ser atacada por más de un saqueador al mismo tiempo. Además, ControlSimulacion actúa como punto común para la pausa y reanudación de todos los hilos, mientras que LoggerFederacion centraliza el registro de eventos relevantes.

## 4.3 Detalle del módulo distribuido



La Figura 3 recoge la estructura del módulo distribuido basado en RMI. La interfaz `InterfazMonitorRemoto` define el contrato remoto que el servidor pone a disposición del cliente. `MonitorRemotoImpl` implementa esa interfaz y obtiene la información directamente de `EstadoGalaxia`, `SincronizacionRecursos` y `ControlSimulacion`, construyendo los objetos `EstadoRecursosDTO` y `EstadoZonasDTO` que se envían al cliente. Por su parte, `ClienteRemotoControlador` localiza el objeto remoto en el registro RMI y actualiza la interfaz del cliente (`VentanaClienteRemoto` y `PanelEstadoRemoto`) de forma automática.

## 5. Diagrama de clases

### 5.1 Decisiones de modelado orientado a objetos

Una de las primeras decisiones fue definir `Zona` y `Actor` como clases abstractas en lugar de interfaces. Esto permitió centralizar atributos comunes (nombre, capacidad, estado, flag de recuperación) evitando duplicidad de código en las subclases. La alternativa de usar interfaces habría obligado a reescribir esa lógica en cada tipo concreto, lo que habría dificultado el mantenimiento a medida que el sistema crecía.

`EstadoGalaxia` se implementó como Singleton porque representa el estado global único de la simulación: las listas de actores activos, las zonas del mapa y los contadores de identificadores. Tener más de una instancia habría generado inconsistencias inmediatas entre hilos, ya que cada uno habría trabajado sobre un estado parcial diferente. El uso de

AtomicInteger para los contadores de ID garantiza además que la generación de identificadores únicos sea segura en entornos concurrentes sin necesidad de sincronización explícita en ese punto.

El enumerado TipoRecurso se eligió frente a constantes enteras o cadenas de texto porque aporta seguridad en tiempo de compilación: el compilador detecta cualquier uso incorrecto de un tipo de recurso antes de ejecutar la simulación. Esto simplificó considerablemente la lógica de selección de planetas y depósitos en HiloDelegado, donde el tipo de recurso actúa como clave de emparejamiento entre planeta y depósito orbital.

## *5.2 Decisiones de sincronización y concurrencia*

El mecanismo principal de acceso a zonas se implementó mediante Semaphore en SincronizacionZonas, uno por cada zona, inicializado con su capacidad máxima. La razón es que un semáforo modela de forma natural el concepto de aforo: permite que varios hilos accedan simultáneamente hasta un límite definido, sin necesidad de que compitan en exclusión mutua total. Usar synchronized o un ReentrantLock en su lugar habría bloqueado la zona entera para un único hilo a la vez, lo que no refleja la semántica real del sistema.

Para las operaciones sobre los depósitos de recursos se optó por ReentrantLock con dos Condition diferenciadas: hayEspacio y hayRecursos. Este diseño permite que los hilos delegado esperen de forma selectiva según lo que necesiten (espacio para depositar o unidades para recoger) sin despertar a todos los hilos bloqueados cuando solo una condición cambia. Habría sido más sencillo usar un único wait/notifyAll, pero eso genera despertares innecesarios que degradan el rendimiento cuando el número de hilos activos es alto.

ControlSimulacion centraliza la pausa y reanudación global de la simulación mediante un ReentrantLock y una Condition. Todos los hilos llaman periódicamente a comprobarPausa() dentro de sus bucles principales, lo que garantiza que la pausa sea cooperativa: ningún hilo queda bloqueado a mitad de una sección crítica, sino que termina su operación actual antes de suspenderse. Esto evita deadlocks derivados de pausar un hilo mientras mantiene un lock adquirido.

## *5.3 Decisiones sobre el módulo distribuido*

La comunicación entre el servidor de simulación y los clientes se implementó mediante RMI (Remote Method Invocation) en lugar de sockets puros. RMI permite exponer métodos del servidor como si fueran llamadas locales desde el cliente, lo que simplifica considerablemente el código del lado cliente: no es necesario gestionar protocolos de serialización propios, flujos de bytes ni control de conexión. La contrapartida es una menor flexibilidad frente a sockets, pero dado que la comunicación en este sistema es relativamente sencilla (consulta de estado y control básico), RMI resultó la opción más adecuada.

Para el intercambio de datos entre servidor y clientes se definieron los DTOs EstadoRecursosDTO y EstadoZonasDTO, ambos serializables. El uso de objetos de transferencia específicos en lugar de exponer directamente las entidades del modelo evita dos problemas: por un lado, que el cliente reciba más información de la necesaria; por otro, que cambios internos en

el modelo obliguen a modificar la interfaz remota. Los DTOs actúan así como una capa de desacoplamiento entre la lógica de la simulación y la representación que se envía a los clientes.

Un aspecto a destacar es que el servidor lanza el registro RMI de forma embebida en el propio proceso de simulación. Esto simplifica el despliegue (no hace falta arrancar rmiregistry como proceso externo), aunque implica que si el servidor cae, el registro también desaparece y los clientes conectados pierden la referencia remota. Para un entorno de producción real esto sería un punto de mejora, pero en el contexto de esta práctica resulta suficiente y coherente con los requisitos.

## *5.4 Limitaciones, problemas encontrados y posibles mejoras*

La principal limitación del sistema es la ausencia de persistencia: cuando la simulación termina, todo el estado se pierde. No hay mecanismo de guardado ni de reanudación desde un punto anterior. En una versión más completa, sería razonable serializar periódicamente EstadoGalaxia a disco o integrarlo con una base de datos ligera.

Durante el desarrollo, el problema más recurrente fue la aparición de condiciones de carrera en el acceso concurrente a las listas de actores de EstadoGalaxia. En varios puntos del código, un hilo podía iterar sobre la lista de delegados mientras otro la modificaba simultáneamente, lo que lanzaba ConcurrentModificationException. La solución fue proteger esas operaciones con el lock de EstadoGalaxia y, en los casos donde era necesario iterar, trabajar sobre una copia de la lista en lugar de sobre la referencia directa.

Otra limitación detectada es que la interfaz gráfica del servidor solo refleja el estado en tiempo real sin histórico de eventos. Un log persistente con filtrado por tipo de evento o por actor mejoraría considerablemente la capacidad de diagnóstico durante la simulación. Como mejora futura, también podría plantearse hacer configurable el número inicial de actores y los umbrales de recursos desde la propia interfaz, en lugar de tenerlos definidos como constantes en el código.

## **Anexo - Código fuente**



## *Paquete Concurrency*

### *ControlSimulacion.java*

```
package Concurrency;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Controla el ciclo de vida global de la simulacion
 *
 * Todos los hilos activos (delegados, patrullas, saqueadores y generadores) llaman a comprobarPausa()
 * Tambien controla la señal de parada total (simulacionActiva) para que los hilos terminen limpiamente cuando se termina
 * el programa
 */
public class ControlSimulacion {

    private final Lock lock;
    private final Condition reanudada;

    private boolean pausada;
    private boolean simulacionActiva;

    public ControlSimulacion() {
        this.lock = new ReentrantLock();
        this.reanudada = lock.newCondition();
        this.pausada = false;
        this.simulacionActiva = false;
    }

    // =====
    // ARRANQUE Y PARADA
    // =====

    //Inicia la simulacion
    public void iniciar() {
        lock.lock();
        try {
            simulacionActiva = true;
            pausada = false;
        }
    }
}
```

```

        reanudada.signalAll();
    } finally {
        lock.unlock();
    }
}

//Detiene la simulacion
public void detener() {
    lock.lock();
    try {
        simulacionActiva = false;
        pausada = false;
        // Desbloquea todos los hilos para que puedan terminar
        reanudada.signalAll();
    } finally {
        lock.unlock();
    }
}

// =====
// PAUSA Y REANUDACION
// =====

//Pausa la simulacion
public void pausar() {
    lock.lock();
    try {
        pausada = true;
    } finally {
        lock.unlock();
    }
}

//Reanuda la simulacion pausada
public void reanudar() {
    lock.lock();
    try {
        pausada = false;
        reanudada.signalAll();
    } finally {
        lock.unlock();
    }
}

```

```

//Punto de control que cada hilo llama en los momentos seguros de su ciclo (entre fases, antes de dormir o al cambiar
de zona)
//Si la simulacion esta pausada, el hilo se bloquea hasta que se reanude
public void comprobarPausa() throws InterruptedException {
    lock.lock();
    try {
        while (pausada) {
            reanudada.await();
        }
    } finally {
        lock.unlock();
    }
}

// =====
// CONSULTAS DE ESTADO
// =====

//Indica si la simulacion esta activa (no detenida)
public boolean isSimulacionActiva() {
    lock.lock();
    try {
        return simulacionActiva;
    } finally {
        lock.unlock();
    }
}

//Indica si la simulacion esta pausada en el momento
public boolean isPausada() {
    lock.lock();
    try {
        return pausada;
    } finally {
        lock.unlock();
    }
}
}

```

## *GeneradorDelegados.java*

```

package Concurrencia;

import Log.LoggerFederacion;
import Modelo.DelegadoComercial;
import Modelo.EstadoGalaxia;

/**
 * Hilo generador de delegados
 *
 * Responsabilidades:
 * 1. Crear 10 delegados iniciales
 * 2. Vigilar si algun deposito esta por debajo del umbral de alerta
 * 3. Si se cumple la condicion y hay menos de 20 delegados, crear un nuevo delegado y lanzar su hilo
 *
 * Intervalo comprobacion: 5s
 * Max delegados: 20
 */
public class GeneradorDelegados extends Thread {

    private static final int DELEGADOS_INICIALES = 10;
    private static final int MAX_DELEGADOS_ACTIVOS = 20;
    private static final int INTERVALO_COMPROBACION_MS = 5000;

    private final EstadoGalaxia estado;
    private final SincronizacionZonas sincZonas;
    private final SincronizacionRecursos sincRecursos;
    private final ControlSimulacion control;
    private final LoggerFederacion logger;

    public GeneradorDelegados(SincronizacionZonas sincZonas, SincronizacionRecursos sincRecursos, ControlSimulacion control, LoggerFederacion logger) {
        this.estado = EstadoGalaxia.getInstancia();
        this.sincZonas = sincZonas;
        this.sincRecursos = sincRecursos;
        this.control = control;
        this.logger = logger;
        setName("GeneradorDelegados");
        setDaemon(true);
    }

    @Override
    public void run() {
        // --- Fase 1: delegados iniciales ---

```

```

for (int i = 0; i < DELEGADOS_INICIALES; i++) {
    crearYLanzarDelegado();
}
logger.registrarEvento("GeneradorDelegados: " + DELEGADOS_INICIALES + " delegados iniciales creados.");

// --- Fase 2: vigilancia de umbrales ---
while (control.isSimulacionActiva()) {
    try {
        control.comprobarPausa();
        Thread.sleep(INTERVALO_COMPROBACION_MS);
        control.comprobarPausa();

        if (sincRecursos.hayDepositoBajoUmbral()) {
            int activos = estado.getNumeroDelegadosActivos();
            if (activos < MAX_DELEGADOS_ACTIVOS) {
                crearYLanzarDelegado();
                logger.registrarEvento("GeneradorDelegados: nuevo delegado creado por umbral. " + "Total activos: " +
(activos + 1));
            }
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        break;
    }
}

//Crea un nuevo delegado, lo registra en el estado global y lanza su hilo
private void crearYLanzarDelegado() {
    String id = estado.generarIdDelegado();
    DelegadoComercial delegado = new DelegadoComercial(id);
    estado.agregarDelegado(delegado);

    HiloDelegado hilo = new HiloDelegado(delegado, sincZonas, sincRecursos, control, logger);
    hilo.start();
}
}

```

## *GeneradorPatrullas.java*

```

package Concurrencia;

import Log.LoggerFederacion;
import Modelo.EstadoGalaxia;
import Modelo.PatrullaFederal;

/**
 * Hilo generador de patrullas
 *
 * Responsabilidades:
 * 1. Crear las 2 patrullas iniciales
 * 2. Vigilar si hay 3 o mas saqueadores activos y menos de 5 patrullas en servicio
 * 3. Si se cumple la condicion, crear una nueva patrulla
 *
 * Intervalo de comprobacion: 8s
 * Limite de patrullas: 5
 * Saqueadores minimos para generar refuerzo: 3
 */
public class GeneradorPatrullas extends Thread {

    private static final int PATRULLAS_INICIALES = 2;
    private static final int MAX_PATRULLAS_ACTIVAS = 5;
    private static final int UMBRAL_SAKEADORES = 3;
    private static final int INTERVALO_COMPROBACION_MS = 8000;

    private final EstadoGalaxia estado;
    private final SincronizacionZonas sincZonas;
    private final SincronizacionSaqueadores sincSaqueadores;
    private final ControlSimulacion control;
    private final LoggerFederacion logger;

    public GeneradorPatrullas(SincronizacionZonas sincZonas, SincronizacionSaqueadores sincSaqueadores,
        ControlSimulacion control, LoggerFederacion logger) {
        this.estado = EstadoGalaxia.getInstance();
        this.sincZonas = sincZonas;
        this.sincSaqueadores = sincSaqueadores;
        this.control = control;
        this.logger = logger;
        setName("GeneradorPatrullas");
        setDaemon(true);
    }

    @Override
    public void run() {

```

```

// --- Fase 1: patrullas iniciales ---
for (int i = 0; i < PATRULLAS_INICIALES; i++) {
    crearYLanzarPatrulla();
}
logger.registrarEvento("GeneradorPatrullas: " + PATRULLAS_INICIALES + " patrullas iniciales creadas.");

// --- Fase 2: vigilancia de refuerzos ---
while (control.isSimulacionActiva()) {
    try {
        control.comprobarPausa();
        Thread.sleep(INTERVALO_COMPROBACION_MS);
        control.comprobarPausa();

        int numSaqueadores = estado.getNumeroSaqueadores();
        int numPatrullas = estado.getNumeroPatrullas();

        if (numSaqueadores >= UMBRAL_SAKEADORES && numPatrullas < MAX_PATRULLAS_ACTIVAS) {
            crearYLanzarPatrulla();
            logger.registrarEvento("GeneradorPatrullas: refuerzo creado. " + "Saqueadores activos: " + numSaqueadores + "
| Patrullas activas: " + (numPatrullas + 1));
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        break;
    }
}

//Crea nueva patrulla, la registra en el estado global y lanza su hilo
private void crearYLanzarPatrulla() {
    String id = estado.generarIdPatrulla();
    PatrullaFederal patrulla = new PatrullaFederal(id);
    estado.agregarPatrulla(patrulla);

    HiloPatrulla hilo = new HiloPatrulla(patrulla, sincZonas, sincSaqueadores, control, logger);
    hilo.start();
}
}

```

## *GeneradorSaqueadores.java*

```
package Concurrencia;
```

```

import Log.LoggerFederacion;
import Modelo.EstadoGalaxia;
import Modelo.Saqueador;

/**
 * Hilo generador de saqueadores
 *
 * Genera un nuevo saqueador cada 10-20s), siempre que no existan mas de 40 saqueadores
 * El primer saqueador no se genera directamente, espera al intervalo
 */
public class GeneradorSaqueadores extends Thread {

    private static final int MAX_SAQUEADORES = 40;
    private static final int INTERVALO_MIN_MS = 10000;
    private static final int INTERVALO_RANGO_MS = 10000; // 10000 + random(10000)

    private final EstadoGalaxia estado;
    private final SincronizacionZonas sincZonas;
    private final SincronizacionRecursos sincRecursos;
    private final SincronizacionSaqueadores sincSaqueadores;
    private final ControlSimulacion control;
    private final LoggerFederacion logger;

    public GeneradorSaqueadores(SincronizacionZonas sincZonas, SincronizacionRecursos sincRecursos,
    SincronizacionSaqueadores sincSaqueadores, ControlSimulacion control, LoggerFederacion logger) {
        this.estado = EstadoGalaxia.getInstance();
        this.sincZonas = sincZonas;
        this.sincRecursos = sincRecursos;
        this.sincSaqueadores = sincSaqueadores;
        this.control = control;
        this.logger = logger;
        setName("GeneradorSaqueadores");
        setDaemon(true);
    }

    @Override
    public void run() {
        while (control.isSimulacionActiva()) {
            try {
                // Espera aleatoria entre 10 y 20s
                long espera = INTERVALO_MIN_MS + (long)(Math.random() * INTERVALO_RANGO_MS);
                Thread.sleep(espera);
                control.comprobarPausa();
            }
        }
    }
}

```



```

        int numActuales = estado.getNumeroSaqueadores();
        if (numActuales < MAX_SAQUEADORES) {
            crearYLanzarSaqueador();
            logger.registrarEvento("GeneradorSaqueadores: nuevo saqueador creado." + "Total activos: " + (numActuales +
1));
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        break;
    }
}
}

//Crea un saqueador, lo registra en el estado global y lanza su hilo
private void crearYLanzarSaqueador() {
    String id = estado.generarIdSaqueador();
    Saqueador saqueador = new Saqueador(id);
    estado.agregarSaqueador(saqueador);

    HiloSaqueador hilo = new HiloSaqueador(
        saqueador, sincZonas, sincRecursos,
        sincSaqueadores, control, logger);
    hilo.start();
}
}

```

## *HiloDelegado.java*

```

package Concurrencia;

import Log.LoggerFederacion;
import Modelo.DelegadoComercial;
import Modelo.DepositoOrbital;
import Modelo.EstadoGalaxia;
import Modelo.PlanetaMinero;
import Modelo.TipoRecurso;
import Modelo.Zona;

import java.util.List;

/**
 * Ejecuta el ciclo de vida de un delegado

```

```

*
* Ciclo:
* 1. Esperar en el Centro de Coordinacion (2-4s)
* 2. Seleccionar aleatoriamente un planeta dependiendo del recurso
* 3. Viajar al planeta y extraer recursos (3-7s, entre 10-30uds)
* 4. Depositar en deposito orbital
* 5. Volver al centro y repetir
*
* Si es expulsado durante un ataque:
* -> Zona de recuperacion (5-15s)
* -> Volver al centro y continuar
*/
public class HiloDelegado extends Thread {

    // Tiempos en ms
    private static final int T_ESPERA_CENTRO_MIN = 2000;
    private static final int T_ESPERA_CENTRO_RANGO = 2000;
    private static final int T_EXTRACCION_MIN = 3000;
    private static final int T_EXTRACCION_RANGO = 4000;
    private static final int T_DEPOSITO_MIN = 1000;
    private static final int T_DEPOSITO_RANGO = 2000;
    private static final int T_RECUPERACION_MIN = 5000;
    private static final int T_RECUPERACION_RANGO = 10000;

    // Unidades extraidas por ciclo
    private static final int EXTRACCION_MIN = 10;
    private static final int EXTRACCION_RANGO = 21; // (10, 30)

    private final DelegadoComercial delegado;
    private final EstadoGalaxia estado;
    private final SincronizacionZonas sincZonas;
    private final SincronizacionRecursos sincRecursos;
    private final ControlSimulacion control;
    private final LoggerFederacion logger;

    public HiloDelegado(DelegadoComercial delegado, SincronizacionZonas sincZonas, SincronizacionRecursos
sincRecursos, ControlSimulacion control, LoggerFederacion logger) {
        this.delegado = delegado;
        this.estado = EstadoGalaxia.getInstancia();
        this.sincZonas = sincZonas;
        this.sincRecursos = sincRecursos;
        this.control = control;
        this.logger = logger;
        setName("Hilo-" + delegado.getId());
    }

```

```

}

@Override
public void run() {
    logger.registrarEvento(delegado.getId() + " comienza su actividad como delegado comercial.");

    while (control.isSimulacionActiva()) {
        try {
            control.comprobarPausa();

            // -----
            // FASE 1: Esperar en el Centro de Coordinacion
            // -----
            delegado.setZonaActual(estado.getCentroCoordinacion());
            delegado.setEstado("En Centro de Coordinación");
            logger.registrarEvento(delegado.getId() + " espera en el Centro de Coordinación Federal.");

            long tEspera = T_ESPERA_CENTRO_MIN + (long)(Math.random() * T_ESPERA_CENTRO_RANGO);
            Thread.sleep(tEspera);
            control.comprobarPausa();

            // -----
            // FASE 2: Seleccionar planeta objetivo
            // -----
            TipoRecurso recurso = seleccionarRecursoAleatorio();
            delegado.setRecursoObjetivo(recurso);
            PlanetaMinero planeta = seleccionarPlaneta(recurso);

            // -----
            // FASE 3: Extraer recursos del planeta
            // -----
            delegado.setEstado("Viajando a " + planeta.getNombre());
            logger.registrarEvento(delegado.getId() + " viaja al planeta " + planeta.getNombre() + " a extraer " + recurso + ".");

            sincZonas.entrarZona(planeta);
            delegado.setZonaActual(planeta);
            delegado.setEstado("Extrayendo en " + planeta.getNombre());

            try {
                long tExtraccion = T_EXTRACCION_MIN + (long)(Math.random() * T_EXTRACCION_RANGO);
                Thread.sleep(tExtraccion);
                control.comprobarPausa();

                // Comprobar si fue expulsado durante la extraccion
            }

```

```

        if (delegado.isExpulsado()) {
            manejarExpulsion(planeta);
            continue;
        }

        int unidades = EXTRACCION_MIN + (int)(Math.random() * EXTRACCION_RANGO);
        delegado.setUnidadesTransportadas(unidades);

        logger.registrarEvento(delegado.getId() + " ha extraído " + unidades + " u. de " + recurso + " en " +
            planeta.getNombre() + ".");
    } finally {
        sincZonas.salirZona(planeta);
    }

    control.comprobarPausa();

    // -----
    // FASE 4: Depositar en el deposito
    // -----
    DepositoOrbital deposito = getDeposito(recurso);
    delegado.setEstado("Viajando a " + deposito.getNombre());
    logger.registrarEvento(delegado.getId() + " viaja al " + deposito.getNombre() + " con " +
        delegado.getUnidadesTransportadas() + " u. de " + recurso + ".");

    sincZonas.entrarZona(deposito);
    delegado.setZonaActual(deposito);
    delegado.setEstado("Depositando en " + deposito.getNombre());

    try {
        long tDeposito = T_DEPOSITO_MIN + (long)(Math.random() * T_DEPOSITO_RANGO);
        Thread.sleep(tDeposito);
        control.comprobarPausa();

        if (delegado.isExpulsado()) {
            manejarExpulsion(deposito);
            continue;
        }

        sincRecursos.depositar(recurso, delegado.getUnidadesTransportadas());
        logger.registrarEvento(delegado.getId() + " ha depositado " + delegado.getUnidadesTransportadas() + " u. de " +
            recurso + " en " + deposito.getNombre() + ".");
        delegado.setUnidadesTransportadas(0);
    } finally {
        sincZonas.salirZona(deposito);
    }

```

```

    }

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        break;
    }
}

logger.registrarEvento(delegado.getId() + " finaliza su actividad.");
}

// =====
// METODOS AUXILIARES
// =====

//Gestiona expulsion del delegado enviandolo a recuperacion
//Depende de quien lo expulsó (5-15s)
private void manejarExpulsion(Zona zonaActual) throws InterruptedException {
    delegado.setExpulsado(false);
    delegado.setUnidadesTransportadas(0);
    delegado.setZonaActual(estado.getZonaRecuperacion());
    delegado.setEstado("Recuperándose");
    delegado.setEnRecuperacion(true);

    logger.registrarEvento(delegado.getId() + " ha sido expulsado de " + zonaActual.getNombre() + " y va a la Zona de
Recuperación.");

    long tRecuperacion = T_RECUPERACION_MIN + (long)(Math.random() * T_RECUPERACION_RANGO);
    Thread.sleep(tRecuperacion);

    delegado.setEnRecuperacion(false);
    delegado.setEstado("Volviendo al Centro");
}

//Selecciona aleatoriamente uno de los tres tipos de recurso
private TipoRecurso seleccionarRecursoAleatorio() {
    TipoRecurso[] tipos = TipoRecurso.values();
    return tipos[(int)(Math.random() * tipos.length)];
}

//Selecciona aleatoriamente un planeta que produzca el tipo de recurso indicado
private PlanetaMinero seleccionarPlaneta(TipoRecurso recurso) {
    List<PlanetaMinero> candidatos = new java.util.ArrayList<>();
    for (PlanetaMinero p : estado.getPlanetas()) {

```

```

        if (p.getTipoRecurso() == recurso) {
            candidatos.add(p);
        }
    }
    return candidatos.get((int)(Math.random() * candidatos.size()));
}

//Devuelve el deposito correspondiente al recurso
private DepositoOrbital getDeposito(TipoRecurso recurso) {
    switch (recurso) {
        case CRISTAL: return estado.getDepositoCristal();
        case MINERAL: return estado.getDepositoMineral();
        case PLASMA: return estado.getDepositoPlasma();
        default: throw new IllegalArgumentException("Recurso desconocido: " + recurso);
    }
}
}

```

## HiloPatrulla.java

```
package Concurrency;
```

```

import Log.LoggerFederacion;
import Modelo.EstadoGalaxia;
import Modelo.PatrullaFederal;
import Modelo.Saqueador;
import Modelo.Zona;

```

```

import java.util.ArrayList;
import java.util.List;

```

```

/**
 * Hilo que ejecuta el ciclo de vida de una patrulla
 *
 * Ciclo:
 * 1. Preparar sistemas en el Hangar (3-6s)
 * 2. Seleccionar aleatoriamente una zona a patrullar
 * 3. Patrullar la zona (4-8s)
 * 4. Si hay saqueador activo en la zona -> combatir (50/50):
 *    - Victoria: saqueador interrumpido, delegados expulsados, saqueador va a la base (20s penalizacion)
 *    - Derrota: patrulla va al Hangar (8-12 recuperacion)
 * 5. Volver al paso 2 si sigue activa
 */

```

```

public class HiloPatrulla extends Thread {

    private static final int T_HANGAR_MIN = 3000;
    private static final int T_HANGAR_RANGO = 3000;
    private static final int T_PATRULLA_MIN = 4000;
    private static final int T_PATRULLA_RANGO = 4000;
    private static final int T_RECUPERACION_MIN = 8000;
    private static final int T_RECUPERACION_RANGO = 4000;

    private final PatrullaFederal patrulla;
    private final EstadoGalaxia estado;
    private final SincronizacionZonas sincZonas;
    private final SincronizacionSaqueadores sincSaqueadores;
    private final ControlSimulacion control;
    private final LoggerFederacion logger;

    public HiloPatrulla(PatrullaFederal patrulla, SincronizacionZonas sincZonas, SincronizacionSaqueadores
sincSaqueadores, ControlSimulacion control, LoggerFederacion logger) {
        this.patrulla = patrulla;
        this.estado = EstadoGalaxia.getInstancia();
        this.sincZonas = sincZonas;
        this.sincSaqueadores = sincSaqueadores;
        this.control = control;
        this.logger = logger;
        setName("Hilo-" + patrulla.getId());
    }

    @Override
    public void run() {
        logger.registrarEvento(patrulla.getId() + " comienza su actividad como patrulla federal.");

        try {
            // --- Fase inicial: preparar sistemas en el hangar ---
            patrulla.setZonaActual(estado.getHangarPatrullas());
            patrulla.setEstado("Preparando sistemas en Hangar");
            logger.registrarEvento(patrulla.getId() + " prepara sus sistemas en el Hangar de Patrullas.");

            long tHangar = T_HANGAR_MIN + (long)(Math.random() * T_HANGAR_RANGO);
            Thread.sleep(tHangar);

            // --- Ciclo principal de patrullaje ---
            while (control.isSimulacionActiva()) {
                control.comprobarPausa();
            }
        }
    }
}

```

```

//FASE 1: Seleccionar zona a patrullar
Zona zonaObjetivo = seleccionarZonaAleatoria();
patrulla.setEstado("Viajando a " + zonaObjetivo.getNombre());
logger.registrarEvento(patrulla.getId() + " se dirige a patrullar " + zonaObjetivo.getNombre() + ".");

//FASE 2: Entrar y patrullar la zona
sincZonas.entrarZona(zonaObjetivo);
patrulla.setZonaActual(zonaObjetivo);
patrulla.setEstado("Patrullando " + zonaObjetivo.getNombre());

try {
    long tPatrulla = T_PATRULLA_MIN + (long)(Math.random() * T_PATRULLA_RANGO);
    Thread.sleep(tPatrulla);
    control.comprobarPausa();

    //FASE 3: Comprobar si hay saqueador en esa zona
    if (sincSaqueadores.estiendiendoAtacada(zonaObjetivo)) {
        combatir(zonaObjetivo);
        // Si fue derrotada, salir del try para ir al hangar
        if (patrulla.getEstado().equals("Derrotada")) {
            sincZonas.salirZona(zonaObjetivo);
            irARrecuperacionHangar();
            continue;
        }
    }
} finally {
    //Solo liberar si no fue liberada en combate
    if (!patrulla.getEstado().equals("Derrotada") && patrulla.getZonaActual() == zonaObjetivo) {
        sincZonas.salirZona(zonaObjetivo);
    }
}
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}

logger.registrarEvento(patrulla.getId() + " finaliza su actividad.");
}

// =====
// METODOS AUXILIARES
// =====

//Resuelve el combate entre la patrulla y el saqueador activo en la zona indicada. Probabilidad 50/50 para cada bando

```



```

private void combatir(Zona zona) throws InterruptedException {
    // Buscar el saqueador activo en la zona
    Saqueador objetivo = buscarSaqueadorEnZona(zona);
    if (objetivo == null) return;

    patrulla.setEnCombate(true);
    objetivo.setFaseActual(Saqueador.FaseAtaque.EN_COMBATE);
    logger.registrarEvento(patrulla.getId() + " inicia combate con " + objetivo.getId() + " en " + zona.getNombre() + ".");

    boolean victoriaPatrulla = Math.random() < 0.5;

    if (victoriaPatrulla) {
        // --- Victoria de la patrulla ---
        patrulla.registrarVictoria();
        patrulla.setEnCombate(false);
        patrulla.setEstado("Patrullando " + zona.getNombre());

        // El saqueador es derrotado: libera la zona y va a la base
        sincSaqueadores.finalizarAtaque(zona);
        objetivo.setFaseActual(Saqueador.FaseAtaque.EN_BASE);
        objetivo.setEstado("Derrotado - recuperándose en Base");

        // Expulsar delegados que estuvieran en la zona
        expulsarDelegadosDeZona(zona);

        logger.registrarEvento(patrulla.getId() + " derrota a " + objetivo.getId() + " en " + zona.getNombre() + ". Delegados de la zona expulsados.");
    } else {
        // --- Derrota de la patrulla ---
        patrulla.registrarDerrota();
        patrulla.setEnCombate(false);
        patrulla.setEstado("Derrotada");

        logger.registrarEvento(patrulla.getId() + " es derrotada por " + objetivo.getId() + " en " + zona.getNombre() + ". Va al Hangar a recuperarse.");
    }
}

//Envia la patrulla derrotada al hangar a recuperarse (8-12s)
private void irARecuperacionHangar() throws InterruptedException {
    patrulla.setZonaActual(estados.getHangarPatrullas());
    patrulla.setEstado("Recuperándose en Hangar");
    patrulla.setEnRecuperacion(true);
}

```

```

        logger.registrarEvento(patrulla.getId() + " se recupera en el Hangar de Patrullas.");

        long tRec = T_RECUPERACION_MIN + (long)(Math.random() * T_RECUPERACION_RANGO);
        Thread.sleep(tRec);

        patrulla.setEnRecuperacion(false);
        patrulla.setEstado("Listo");
    }

    //Expulsa todos los delegados presentes en la zona atacada, e iran a la Zona de Recuperacion.
    private void expulsarDelegadosDeZona(Zona zona) {
        for (Modelo.DelegadoComercial d : estado.getDelegados()) {
            if (d.getZonaActual() != null && d.getZonaActual().equals(zona) && !d.isExpulsado()) {
                d.setExpulsado(true);
            }
        }
    }

    //Busca el saqueador activo (en fase SAQUEANDO o DESPLAZANDOSE) cuya zona objetivo coincide con la zona indicada
    private Saqueador buscarSaqueadorEnZona(Zona zona) {
        for (Saqueador s : estado.getSaqueadores()) {
            if (zona.equals(s.getObjetivo()) && s.estaAtacando()) {
                return s;
            }
        }
        return null;
    }

    //Selecciona aleatoriamente una zona entre los planetas y depositos del sistema
    private Zona seleccionarZonaAleatoria() {
        List<Zona> zonas = new ArrayList<>();
        zonas.addAll(estado.getPlanetas());
        zonas.addAll(estado.getDepositos());
        return zonas.get((int)(Math.random() * zonas.size()));
    }
}

```

## HiloSaqueador.java

```

package Concurrency;

import Log.LoggerFederacion;

```

```

import Modelo.DepositoOrbital;
import Modelo.EstadoGalaxia;
import Modelo.PlanetaMinero;
import Modelo.Saqueador;
import Modelo.Zona;

import java.util.List;

/**
 * Hilo que ejecuta el ciclo de vida de un saqueador espacial
 *
 * Ciclo:
 * 1. Esperar en la Base (3-6s)
 * 2. Seleccionar objetivo:
 *    - 70% -> depósito orbital aleatorio
 *    - 30% -> planeta minero aleatorio
 * 3. Desplazarse al objetivo (2-5s)
 * 4. Intentar el ataque:
 *    - Si hay patrulla -> combate (50/50):
 *      Victoria saqueador: expulsa delegados, saquea recursos
 *      Derrota saqueador: va a la base (20s penalizacion)
 *    - Sin patrulla -> saquea directamente
 * 5. Esperar en la base (10s obligatorios entre ataques)
 * 6. Repetir
 */
public class HiloSaqueador extends Thread {

    private static final int T_BASE_MIN = 3000;
    private static final int T_BASE_RANGO = 3000;
    private static final int T_DESPLAZAMIENTO_MIN = 2000;
    private static final int T_DESPLAZAMIENTO_RANGO = 3000;
    private static final int T_SAQUEO_MIN = 2000;
    private static final int T_SAQUEO_RANGO = 3000;
    private static final int T_ESPERA_ENTRE_ATAQUES_MS = 10000;
    private static final int T_PENALIZACION_DERROTA_MS = 20000;

    private static final double PROB_DEPOSITO = 0.70;

    private final Saqueador saqueador;
    private final EstadoGalaxia estado;
    private final SincronizacionZonas sincZonas;
    private final SincronizacionRecursos sincRecursos;
    private final SincronizacionSaqueadores sincSaqueadores;
    private final ControlSimulacion control;

```

```

private final LoggerFederacion logger;

public HiloSaqueador(Saqueador saqueador, SincronizacionZonas sincZonas, SincronizacionRecursos sincRecursos,
SincronizacionSaqueadores sincSaqueadores, ControlSimulacion control, LoggerFederacion logger) {
    this.saqueador = saqueador;
    this.estado = EstadoGalaxia.getInstancia();
    this.sincZonas = sincZonas;
    this.sincRecursos = sincRecursos;
    this.sincSaqueadores = sincSaqueadores;
    this.control = control;
    this.logger = logger;
    setName("Hilo-" + saqueador.getId());
}

@Override
public void run() {
    logger.registrarEvento(saqueador.getId() + " aparece en la galaxia.");

    try {
        while (control.isSimulacionActiva()) {
            control.comprobarPausa();

            // -----
            // FASE 1: Esperar en la base
            // -----
            saqueador.setZonaActual(estado.getBaseSaqueadores());
            saqueador.setFaseActual(Saqueador.FaseAtaque.EN_BASE);
            saqueador.setEstado("Esperando en Base");

            long tBase = T_BASE_MIN + (long)(Math.random() * T_BASE_RANGO);
            Thread.sleep(tBase);
            control.comprobarPausa();

            // -----
            // FASE 2: Seleccionar objetivo
            // -----
            Zona objetivo = seleccionarObjetivo();
            saqueador.setObjetivo(objetivo);
            saqueador.setFaseActual(Saqueador.FaseAtaque.DESPLAZANDOSE);
            saqueador.setEstado("Viajando a " + objetivo.getNombre());
            logger.registrarEvento(saqueador.getId() + " se dirige a atacar " + objetivo.getNombre() + ".");

            // -----
            // FASE 3: Desplazarse al objetivo

```

```

// -----
long tDespl = T_DESPLAZAMIENTO_MIN + (long)(Math.random() * T_DESPLAZAMIENTO_RANGO);
Thread.sleep(tDespl);
control.comprobarPausa();

// -----
// FASE 4: Intentar el ataque
// -----
objetivo.setBajoAtaque(true);

sincSaqueadores.iniciarAtaque(objetivo);
saqueador.setFaseActual(Saqueador.FaseAtaque.SAQUEANDO);
saqueador.setEstado("Atacando " + objetivo.getNombre());

try {
    //Comprobar si hay patrulla en zona
    PatrullaEnZona resultado = buscarPatrullaEnZona(objetivo);

    if (resultado != null) {
        boolean victoriaSaqueador = Math.random() < 0.5;
        saqueador.setFaseActual(Saqueador.FaseAtaque.EN_COMBATE);
        logger.registrarEvento(saqueador.getId() + " combate con " + resultado.getId() + " en " +
objetivo.getNombre() + ".");

        if (!victoriaSaqueador) {
            // --- Derrota del saqueador ---
            logger.registrarEvento(saqueador.getId() + " es derrotado por " + resultado.getId() + ". Penalización de 20
s en la Base.");
            sincSaqueadores.finalizarAtaque(objetivo);
            saqueador.setFaseActual(Saqueador.FaseAtaque.EN_BASE);
            saqueador.setObjetivo(null);
            saqueador.setZonaActual(estado.getBaseSaqueadores());
            saqueador.setEstado("Derrotado - penalización");
            Thread.sleep(T_PENALIZACION_DERROTA_MS);
            continue;
        }
        // Victoria del saqueador: continua saqueando
        logger.registrarEvento(saqueador.getId() + " vence a " + resultado.getId() + " en " + objetivo.getNombre() +
".");
    }

// -----
// FASE 5: Saquear
// -----

```

```

        expulsarDelegadosDeZona(objetivo);
        saqueador.setFaseActual(Saqueador.FaseAtaque.SAQUEANDO);

        long tSaqueo = T_SAQUEO_MIN + (long)(Math.random() * T_SAQUEO_RANGO);
        Thread.sleep(tSaqueo);

        int robado = ejecutarSaqueo(objetivo);
        saqueador.setUnidadesRobadas(saqueador.getUnidadesRobadas() + robado);

        logger.registrarEvento(saqueador.getId() + " ha saqueado " + robado + " u. de " + objetivo.getNombre() + ".");

    } finally {
        sincSaqueadores.finalizarAtaque(objetivo);
        objetivo.setBajoAtaque(false);
    }

    // -----
    // FASE 6: Regresar a la base y esperar
    // -----
    saqueador.setFaseActual(Saqueador.FaseAtaque.REGRESANDO);
    saqueador.setEstado("Regresando a la Base");
    saqueador.setObjetivo(null);

    Thread.sleep(T_ESPERA_ENTRE_ATAQUES_MS);
    control.comprobarPausa();
}

} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}

estado.eliminarSaqueador(saqueador);
logger.registrarEvento(saqueador.getId() + " abandona la galaxia.");
}

// =====
// METODOS AUXILIARES
// =====

//Selecciona el objetivo del ataque:
// 70% -> deposito
// 30% -> planeta
private Zona seleccionarObjetivo() {
    if (Math.random() < PROB_DEPOSITO) {

```

```

        List<DepositoOrbital> depositos = estado.getDepositos();
        return depositos.get((int)(Math.random() * depositos.size()));
    } else {
        List<PlanetaMinero> planetas = estado.getPlanetas();
        return planetas.get((int)(Math.random() * planetas.size()));
    }
}

//Ejecuta el saqueo sobre la zona objetivo
//Si es deposito: retirar hasta 30% de las unidades
//Si es planeta: interrumpe la extraccion
private int ejecutarSaqueo(Zona zona) throws InterruptedException {
    if (zona instanceof DepositoOrbital) {
        DepositoOrbital deposito = (DepositoOrbital) zona;
        return sincRecursos.vaciarPorSaqueo(deposito.getTipoRecurso());
    }
    return 0;
}

//Expulsa todos los delegados presentes en la zona atacada
private void expulsarDelegadosDeZona(Zona zona) {
    for (Modelo.DelegadoComercial d
        : estado.getDelegados()) {
        if (d.getZonaActual() != null && d.getZonaActual().equals(zona) && !d.isExpulsado()) {
            d.setExpulsado(true);
        }
    }
}

// Busca una patrulla activa en la zona indicada. Devuelve la primera que encuentre, o null si no hay ninguna
private PatrullaEnZona buscarPatrullaEnZona(Zona zona) {
    for (Modelo.PatrullaFederal p
        : estado.getPatrullas()) {
        if (p.getZonaActual() != null && p.getZonaActual().equals(zona) && !p.isEnRecuperacion()) {
            return new PatrullaEnZona(p.getId());
        }
    }
    return null;
}

// DTO (Data Transfer Object) minimo para transportar el ID de la patrulla encontrada
private static class PatrullaEnZona {
    private final String id;
    PatrullaEnZona(String id) {this.id = id;}
}

```

```

        String getId() {return id;}
    }
}

```

## *SincronizacionRecursos.java*

```

package Concurrency;

import Modelo.DepositoOrbital;
import Modelo.EstadoGalaxia;
import Modelo.TipoRecurso;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Gestiona el acceso thread-safe a las unidades almacenadas en los depósitos
 *
 * Protege dos operaciones críticas:
 * depositar(): el delegado añade unidades al depósito
 * vaciarPorSaqueo(): el saqueador retira unidades
 */
public class SincronizacionRecursos {

    private final Lock lock;
    private final Condition hayEspacio;
    private final Condition hayRecursos;

    public SincronizacionRecursos() {
        this.lock = new ReentrantLock();
        this.hayEspacio = lock.newCondition();
        this.hayRecursos = lock.newCondition();
    }

    //Deposita unidades en el depósito correspondiente al tipo de recurso indicado
    //Si no hay espacio suficiente, el hilo espera
    public void depositar(TipoRecurso tipoRecurso, int cantidad)
        throws InterruptedException {
        DepositoOrbital deposito = getDeposito(tipoRecurso);
        lock.lock();
        try {
            //Monitor de Mesa

```



```

        while (!deposito.tieneEspacio(cantidad)) {
            hayEspacio.await();
        }
        deposito.setUnidadesActuales(deposito.getUnidadesActuales() + cantidad);
        //Notificar a posibles saqueadores esperando recursos
        hayRecursos.signalAll();
    } finally {
        lock.unlock();
    }
}

//Retira hasta un 30% de las unidades actuales de un deposito durante un saqueo (min. 1)
//Si el deposito esta vacio, el saqueador espera
public int vaciarPorSaqueo(TipoRecurso tipoRecurso) throws InterruptedException {
    DepositoOrbital deposito = getDeposito(tipoRecurso);
    lock.lock();
    try {
        while (deposito.getUnidadesActuales() == 0) {
            hayRecursos.await();
        }
        int unidadesActuales = deposito.getUnidadesActuales();
        int robadas = Math.max(1, (int)(unidadesActuales * 0.30));
        deposito.setUnidadesActuales(unidadesActuales - robadas);
        // Notificar a delegados esperando espacio
        hayEspacio.signalAll();
        return robadas;
    } finally {
        lock.unlock();
    }
}

//Consulta de unidades actuales de un deposito
//Usado por GUI, cliente remoto y generadores de actores
public int getUnidadesActuales(TipoRecurso tipoRecurso) {
    DepositoOrbital deposito = getDeposito(tipoRecurso);
    lock.lock();
    try {
        return deposito.getUnidadesActuales();
    } finally {
        lock.unlock();
    }
}
}

```

```

//Comprueba si un deposito esta por debajo del umbral de alerta (10% de capacidad max). Usado por
GeneradorDelegados
public boolean hayDepositoBajoUmbral() {
    lock.lock();
    try {
        EstadoGalaxia estado = EstadoGalaxia.getInstancia();
        for (DepositoOrbital d : estado.getDepositos()) {
            if (d.estaPorDebajoUmbral()) {
                return true;
            }
        }
        return false;
    } finally {
        lock.unlock();
    }
}

// --- Auxiliar ---

private DepositoOrbital getDeposito(TipoRecurso tipoRecurso) {
    EstadoGalaxia estado = EstadoGalaxia.getInstancia();
    switch (tipoRecurso) {
        case CRISTAL: return estado.getDepositoCristal();
        case MINERAL: return estado.getDepositoMineral();
        case PLASMA: return estado.getDepositoPlasma();
        default: throw new IllegalArgumentException(
            "Tipo de recurso desconocido: " + tipoRecurso);
    }
}
}

```

## *SincronizacionSaqueadores.java*

```

package Concurrencia;

import Modelo.DepositoOrbital;
import Modelo.EstadoGalaxia;
import Modelo.PlanetaMinero;
import Modelo.Zona;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.Condition;

```

```

import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Garantiza maximo un saqueador atacando una zona en un momento
 * Si la zona objetivo del saqueador esta siendo atacada, espera hasta que quede libre
 */
public class SincronizacionSaqueadores {

    // true = zona siendo atacada actualmente
    private final Map<Zona, Boolean> zonasOcupadas;

    private final Lock lock;
    private final Condition zonaLibre;

    public SincronizacionSaqueadores() {
        this.lock = new ReentrantLock();
        this.zonaLibre = lock.newCondition();
        this.zonasOcupadas = new HashMap<>();

        EstadoGalaxia estado = EstadoGalaxia.getInstancia();

        // Registrar zonas atacables
        for (PlanetaMinero planeta : estado.getPlanetas()) {
            zonasOcupadas.put(planeta, false);
        }
        for (DepositoOrbital deposito : estado.getDepositos()) {
            zonasOcupadas.put(deposito, false);
        }
    }

    //Ataca una zona. Si ya hay un saqueador, bloquea al hilo hasta que quede libre
    public void iniciarAtaque(Zona zona) throws InterruptedException {
        lock.lock();
        try {
            // Monitor de Mesa
            while (zonasOcupadas.getOrDefault(zona, false)) {
                zonaLibre.await();
            }
            zonasOcupadas.put(zona, true);
        } finally {
            lock.unlock();
        }
    }
}

```

```

//Libera la zona al finalizar el ataque
//Notifica a todos los saqueadores que esperan por esa zona
public void finalizarAtaque(Zona zona) {
    lock.lock();
    try {
        zonasOcupadas.put(zona, false);
        zonaLibre.signalAll();
    } finally {
        lock.unlock();
    }
}

//Consulta si una zona esta siendo atacada, usado por HiloPatrulla
public boolean estasiendoAtacada(Zona zona) {
    lock.lock();
    try {
        return zonasOcupadas.getOrDefault(zona, false);
    } finally {
        lock.unlock();
    }
}
}

```

## *SincronizacionZonas.java*

```

package Concurrencia;

import Modelo.EstadoGalaxia;
import Modelo.PlanetaMinero;
import Modelo.DepositoOrbital;
import Modelo.Zona;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.Semaphore;

/**
 * Gestiona el control de acceso por capacidad a todas las zonas de la galaxia mediante semaforos
 * Cada zona con limite (planetas: max 4, depositos: max 3) tiene su propio Semaphore
 *
 * Orden: FIFO
 */

```

```

public class SincronizacionZonas {

    // Mapa zona -> semaforo de capacidad
    private final Map<Zona, Semaphore> semaforosCapacidad;

    public SincronizacionZonas() {
        semaforosCapacidad = new HashMap<>();
        EstadoGalaxia estado = EstadoGalaxia.getInstance();

        // Planetas mineros (capacidad max. 4 cada uno)
        for (PlanetaMinero planeta : estado.getPlanetas()) {
            semaforosCapacidad.put(planeta, new Semaphore(planeta.getCapacidadMaxima(), true));
        }

        // Depositos (capacidad max. 3 cada uno)
        for (DepositoOrbital deposito : estado.getDepositos()) {
            semaforosCapacidad.put(deposito, new Semaphore(deposito.getCapacidadMaxima(), true));
        }
    }

    //Solicita acceso a una zona con limite de capacidad
    //Bloquea al hilo si la zona esta llena hasta que se libere un hueco
    public void entrarZona(Zona zona) throws InterruptedException {
        Semaphore sem = semaforosCapacidad.get(zona);
        if (sem != null) {
            sem.acquire();
        }
    }

    //Libera el hueco ocupado en una zona con límite de capacidad
    public void salirZona(Zona zona) {
        Semaphore sem = semaforosCapacidad.get(zona);
        if (sem != null) {
            sem.release();
        }
    }

    //Devuelve los huecos disponibles en una zona
    //Util para GUI y cliente remoto
    public int getHuecosDisponibles(Zona zona) {
        Semaphore sem = semaforosCapacidad.get(zona);
        if (sem != null) {
            return sem.availablePermits();
        }
    }
}

```

```

        return -1;
    }
}

```

## ***Paquete GUI***

### *PanelActores.java*

```

package GUI;

import Modelo.DelegadoComercial;
import Modelo.EstadoGalaxia;
import Modelo.PatrullaFederal;
import Modelo.Saqueador;

import javax.swing.*;
import javax.swing.border.TitledBorder;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.DefaultTableCellRenderer;
import java.awt.*;
import java.util.List;

//Tablas que muestran en tiempo real el estado de todos los actores activos en la simulacion
public class PanelActores extends JPanel {

    // --- Columnas de cada tabla ---
    private static final String[] COLS_DELEGADO = {"ID", "Zona actual", "Recurso", "Transportando", "Estado"};
    private static final String[] COLS_PATRULLA = {"ID", "Zona actual", "C. Ganados", "C. Perdidos", "Estado"};
    private static final String[] COLS_SAKUEADOR = {"ID", "Zona actual", "Objetivo", "U. robadas", "Estado"};

    private final DefaultTableModel modeloDelegados;
    private final DefaultTableModel modeloPatrullas;
    private final DefaultTableModel modeloSaqueadores;

    private final EstadoGalaxia estado;

    public PanelActores() {
        this.estado = EstadoGalaxia.getInstancia();

        modeloDelegados = crearModelo(COLS_DELEGADO);
    }

```

```

modeloPatrullas = crearModelo(COLS_PATRULLA);
modeloSaqueadores = crearModelo(COLS_SAQUEADOR);

setLayout(new GridLayout(3, 1, 0, 6));
setBorder(BorderFactory.createEmptyBorder(4, 4, 4, 4));

add(crearPanelTabla("Delegados Comerciales", modeloDelegados, new Color(60, 120, 200)));
add(crearPanelTabla("Patrullas Federales", modeloPatrullas, new Color(40, 160, 80)));
add(crearPanelTabla("Saqueadores", modeloSaqueadores, new Color(200, 60, 60)));
}

// =====
// CONSTRUCCION
// =====

private JPanel crearPanelTabla(String titulo, DefaultTableModel modelo, Color colorCabecera) {
    JPanel panel = new JPanel(new BorderLayout());
    panel.setBorder(new TitledBorder(titulo);

    JTable tabla = new JTable(modelo);
    tabla.setFillsViewportHeight(true);
    tabla.setRowHeight(20);
    tabla.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 11));
    tabla.setGridColor(new Color(220, 220, 220));
    tabla.setSelectionBackground(new Color(210, 225, 245));
    tabla.setAutoResizeMode(JTable.AUTO_RESIZE_ALL_COLUMNS);

    // Cabecera con color propio de cada actor
    tabla.getTableHeader().setBackground(colorCabecera);
    tabla.getTableHeader().setForeground(Color.WHITE);
    tabla.getTableHeader().setFont(new Font(Font.SANS_SERIF, Font.BOLD, 11));

    // Renderizador de color en columna "Estado"
    int colEstado = modelo.getColumnCount() - 1;
    tabla.getColumnModel().getColumn(colEstado).setCellRenderer(new EstadoCellRenderer());

    JScrollPane scroll = new JScrollPane(tabla);
    scroll.setPreferredSize(new Dimension(0, 100));
    panel.add(scroll, BorderLayout.CENTER);
    return panel;
}

// =====
// ACTUALIZACION

```

```
// =====

//Refresca las tres tablas con los datos actuales de EstadoGalaxia
public void refrescar() {
    refrescarDelegados();
    refrescarPatrullas();
    refrescarSaqueadores();
}

private void refrescarDelegados() {
    modeloDelegados.setRowCount(0);
    List<DelegadoComercial> lista = estado.getDelegados();
    for (DelegadoComercial d : lista) {
        modeloDelegados.addRow(new Object[]{
            d.getId(),
            d.getZonaActual() != null ? d.getZonaActual().getNombre() : "-",
            d.getRecursoObjetivo() != null ? d.getRecursoObjetivo().name() : "-",
            d.getUnidadesTransportadas(),
            d.getEstado()
        });
    }
}

private void refrescarPatrullas() {
    modeloPatrullas.setRowCount(0);
    List<PatrullaFederal> lista = estado.getPatrullas();
    for (PatrullaFederal p : lista) {
        modeloPatrullas.addRow(new Object[]{
            p.getId(),
            p.getZonaActual() != null
                ? p.getZonaActual().getNombre() : "-",
            p.getCombatesGanados(),
            p.getCombatesPerdidos(),
            p.getEstado()
        });
    }
}

private void refrescarSaqueadores() {
    modeloSaqueadores.setRowCount(0);
    List<Saqueador> lista = estado.getSaqueadores();
    for (Saqueador s : lista) {
        modeloSaqueadores.addRow(new Object[]{
            s.getId(),

```



```

        s.getZonaActual() != null ? s.getZonaActual().getNombre() : "-",
        s.getObjetivo() != null ? s.getObjetivo().getNombre() : "-",
        s.getUnidadesRobadas(),
        s.getEstado()
    });
}
}

// =====
// AUXILIARES
// =====

//Crea un DefaultTableModel no editable con las columnas dadas
private DefaultTableModel crearModelo(String[] columnas) {
    return new DefaultTableModel(columnas, 0) {
        @Override
        public boolean isCellEditable(int row, int col) {
            return false;
        }
    };
}

// =====
// RENDERIZADOR DE COLUMNA ESTADO
// =====

/**
 * Colorea el texto de la celda "Estado" segun su contenido:
 * - Rojo: Derrotado / Expulsado / Atacando
 * - Naranja: Recuperándose / Penalización
 * - Azul: Transportando / Patrullando
 * - Verde: Listo / En Centro
 * - Gris: Resto
 */
private static class EstadoCellRenderer extends DefaultTableCellRenderer {

    @Override
    public Component getTableCellRendererComponent(JTable table, Object value, boolean isSelected, boolean hasFocus,
int row, int column) {

        super.getTableCellRendererComponent(table, value, isSelected, hasFocus, row, column);

        if (value == null) {
            setForeground(Color.GRAY);

```

```

        return this;
    }

    String estado = value.toString().toLowerCase();

    if (estado.contains("derrota") || estado.contains("expulsado") || estado.contains("atacando") ||
estado.contains("saqueando")) {
        setForeground(new Color(190, 40, 40));
    } else if (estado.contains("recuper") || estado.contains("penaliz") || estado.contains("combate")) {
        setForeground(new Color(200, 130, 0));
    } else if (estado.contains("transport") || estado.contains("patrullando") || estado.contains("viajando")) {
        setForeground(new Color(30, 100, 200));
    } else if (estado.contains("listo") || estado.contains("centro") || estado.contains("preparando")) {
        setForeground(new Color(0, 150, 60));
    } else {
        setForeground(Color.DARK_GRAY);
    }
    return this;
}
}
}

```

## PanelLog.java

```

package GUI;

import javax.swing.*;
import javax.swing.border.TitledBorder;
import java.awt.*;

//Muestra en tiempo real los eventos del log
public class PanelLog extends JPanel {

    private static final int MAX_LINEAS = 500;

    private final JTextArea areaLog;
    private final JScrollPane scroll;

    public PanelLog() {
        setLayout(new BorderLayout());
        setBorder(new TitledBorder("Log de Eventos"));

        areaLog = new JTextArea();
    }
}

```

```

areaLog.setEditable(false);
areaLog.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 11));
areaLog.setBackground(new Color(20, 20, 20));
areaLog.setForeground(new Color(180, 230, 180));
areaLog.setLineWrap(true);
areaLog.setWrapStyleWord(false);

scroll = new JScrollPane(areaLog);
scroll.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
scroll.setPreferredSize(new Dimension(0, 160));
add(scroll, BorderLayout.CENTER);

//Boton para limpiar el panel
JButton btnLimpiar = new JButton("Limpiar");
btnLimpiar.setFont(btnLimpiar.getFont().deriveFont(11f));
btnLimpiar.addActionListener(e -> limpiar());
JPanel panelBoton = new JPanel(new FlowLayout(FlowLayout.RIGHT, 4, 2));
panelBoton.add(btnLimpiar);
add(panelBoton, BorderLayout.SOUTH);
}

//Añade una linea al area y hace scroll automatico al final
public void agregarLinea(String linea) {
    // Limitar num de lineas
    if (areaLog.getLineCount() >= MAX_LINES) {
        String texto = areaLog.getText();
        int primerSalto = texto.indexOf("\n");
        if (primerSalto >= 0) {
            areaLog.setText(texto.substring(primerSalto + 1));
        }
    }
    areaLog.append(linea + "\n");
    // Scroll automatico al final
    areaLog.setCaretPosition(areaLog.getDocument().getLength());
}

//Limpia el contenido del area de log visual (no afecta en memoria)
public void limpiar() {
    areaLog.setText("");
}
}

```

### *PanelMapaGalaxia.java*

```

package GUI;

import Modelo.*;

import javax.swing.*;
import java.awt.*;
import java.awt.geom.*;
import java.util.List;

/**
 * Mapa visual de la galaxia
 * Dibuja todas las zonas como nodos con su nombre y numero
 * de actores actuales, con colores distintos por tipo de zona.
 */
public class PanelMapaGalaxia extends JPanel {

    // --- Colores por tipo de zona ---
    private static final Color COLOR_CENTRO = new Color(70, 130, 200);
    private static final Color COLOR_PLANETA = new Color(50, 160, 80);
    private static final Color COLOR_DEPOSITO = new Color(180, 140, 40);
    private static final Color COLOR_HANGAR = new Color(100, 80, 180);
    private static final Color COLOR_BASE_SAQ = new Color(200, 60, 60);
    private static final Color COLOR_RECUPERACION = new Color(140, 140, 140);
    private static final Color COLOR_FONDO = new Color(15, 20, 40);
    private static final Color COLOR_TEXTO = new Color(230, 230, 230);
    private static final Color COLOR_TEXTO_CUENTA = new Color(255, 230, 100);

    // --- Posiciones fijas de cada zona (porcentaje del panel) ---
    // Se calculan en paintComponent() escalando al tamaño real
    private static final double[][] POSICIONES = {
        // {cx%, cy%, radio%} -> una entrada por zona
        {0.50, 0.45, 0.08}, // 0: Centro coordinacion federal
        {0.20, 0.20, 0.07}, // 1: Planeta A
        {0.50, 0.12, 0.07}, // 2: Planeta B
        {0.80, 0.20, 0.07}, // 3: Planeta C
        {0.20, 0.70, 0.07}, // 4: Deposito A
        {0.80, 0.70, 0.07}, // 5: Deposito B
        {0.50, 0.82, 0.07}, // 6: Deposito C
        {0.10, 0.45, 0.06}, // 7: Hangar patrullas
        {0.90, 0.45, 0.06}, // 8: Base saqueadores
        {0.50, 0.92, 0.05}, // 9: Zona recuperacion
    };
}

```

```

private final EstadoGalaxia estado;

public PanelMapaGalaxia() {
    this.estado = EstadoGalaxia.getInstancia();
    setBackground(COLOR_FONDO);
    setPreferredSize(new Dimension(600, 480));
    setToolTipText("Mapa de la galaxia");
}

// =====
// PINTADO
// =====

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
    g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);

    int w = getWidth();
    int h = getHeight();

    dibujarLineasConexion(g2, w, h);
    dibujarZonas(g2, w, h);
    dibujarTitulo(g2, w);
}

// =====
// LINEAS DE CONEXION
// =====

private void dibujarLineasConexion(Graphics2D g2, int w, int h) {
    g2.setColor(new Color(60, 80, 120, 160));
    g2.setStroke(new BasicStroke(1.2f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));

    // Centro -> Planetas (0->1, 0->2, 0->3)
    int[][] conexiones = {
        {0,1},{0,2},{0,3}, // Centro <-> planetas
        {0,4},{0,5},{0,6}, // Centro <-> depositos
        {0,7},{0,8}, // Centro <-> hangar y base
        {1,4},{2,5},{3,6}, // Planetas <-> depositos
        {7,9},{8,9} // Hangar/base <-> Recuperacion
    };
}

```

```

    for (int[] par : conexiones) {
        double[] a = POSICIONES[par[0]];
        double[] b = POSICIONES[par[1]];
        int x1 = (int)(a[0] * w);
        int y1 = (int)(a[1] * h);
        int x2 = (int)(b[0] * w);
        int y2 = (int)(b[1] * h);
        g2.drawLine(x1, y1, x2, y2);
    }
}

// =====
// ZONAS
// =====

private void dibujarZonas(Graphics2D g2, int w, int h) {
    List<Zona> zonas = estado.getTodasLasZonas();

    for (int i = 0; i < zonas.size() && i < POSICIONES.length; i++) {
        Zona zona = zonas.get(i);
        double[] pos = POSICIONES[i];

        int cx = (int)(pos[0] * w);
        int cy = (int)(pos[1] * h);
        int radio = (int)(pos[2] * Math.min(w, h));

        Color colorZona = colorDeZona(zona);
        int actores = contarActoresEnZona(zona);
        int capacidad = zona.getCapacidadMaxima();

        dibujarNodo(g2, cx, cy, radio, colorZona, zona.getNombre(), actores, capacidad, zona);
    }
}

private void dibujarNodo(Graphics2D g2, int cx, int cy, int radio, Color color, String nombre, int actores, int capacidad,
Zona zona) {
    //Sombra
    g2.setColor(new Color(0, 0, 0, 80));
    g2.fillOval(cx - radio + 3, cy - radio + 3, radio * 2, radio * 2);

    //Relleno degradado radial
    RadialGradientPaint grad = new RadialGradientPaint(new Point2D.Float(cx - radio / 3f, cy - radio / 3f), radio * 1.2f,
new float[]{ 0f, 1f }, new Color[]{ color.brighter(), color.darker() });

```

```

g2.setPaint(grad);
g2.fillOval(cx - radio, cy - radio, radio * 2, radio * 2);

//Borde
g2.setColor(color.brighter());
g2.setStroke(new BasicStroke(1.5f));
g2.drawOval(cx - radio, cy - radio, radio * 2, radio * 2);

//Indicador de ocupacion
if (capacidad > 0) {
    float fraccion = Math.min(1f, (float) actores / capacidad);
    g2.setColor(COLOR_TEXTO_CUENTA);
    g2.setStroke(new BasicStroke(3f));
    g2.drawArc(cx - radio, cy - radio, radio * 2, radio * 2, 90, -(int)(fraccion * 360));
}

//Muestra si hay un saqueo en esa zona actualmente
if (zona.isBajoAtaque()) {
    g2.setColor(new Color(255, 60, 60, 100));
    g2.fillOval(cx - radio - 6, cy - radio - 6, (radio + 6) * 2, (radio + 6) * 2);
}

//Nombre de la zona
g2.setColor(COLOR_TEXTO);
g2.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 9));
String[] partes = partirNombre(nombre);
FontMetrics fm = g2.getFontMetrics();
int lineH = fm.getHeight();
int yBase = cy - (partes.length - 1) * lineH / 2;
for (int i = 0; i < partes.length; i++) {
    int xTxt = cx - fm.stringWidth(partes[i]) / 2;
    g2.drawString(partes[i], xTxt, yBase + i * lineH);
}

//Contador de actores bajo el nodo
g2.setColor(COLOR_TEXTO_CUENTA);
g2.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 10));
String cuenta = actores + "/" + capacidad;
fm = g2.getFontMetrics();
g2.drawString(cuenta, cx - fm.stringWidth(cuenta) / 2, cy + radio + fm.getAscent() + 2);
}

private void dibujarTitulo(Graphics2D g2, int w) {
    g2.setColor(new Color(180, 200, 255, 180));

```

```

g2.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 12));
String titulo = "Federación Galáctica de Recursos";
FontMetrics fm = g2.getFontMetrics();
g2.drawString(titulo, (w - fm.stringWidth(titulo)) / 2, 14);
}

// =====
// ACTUALIZACION
// =====

//Fuerza un repintado del mapa con los datos actuales
public void refrescar() {
    repaint();
}

// =====
// AUXILIARES
// =====

private Color colorDeZona(Zona zona) {
    if (zona instanceof CentroCoordinacionFederal)
        return COLOR_CENTRO;
    if (zona instanceof PlanetaMinero)
        return COLOR_PLANETA;
    if (zona instanceof DepositoOrbital)
        return COLOR_DEPOSITO;
    if (zona instanceof HangarPatrullas)
        return COLOR_HANGAR;
    if (zona instanceof BaseSaqueadores)
        return COLOR_BASE_SAQ;
    if (zona instanceof ZonaRecuperacion)
        return COLOR_RECUPERACION;
    return Color.GRAY;
}

//Parte el nombre de una zona en dos lineas si hay mas de 10 caracteres
private String[] partirNombre(String nombre) {
    if (nombre.length() <= 10) return new String[]{ nombre };
    int mitad = nombre.length() / 2;
    int espacio = nombre.lastIndexOf(' ', mitad);
    if (espacio < 0) espacio = nombre.indexOf(' ', mitad);
    if (espacio < 0) return new String[]{ nombre };
    return new String[]{
        nombre.substring(0, espacio),

```



```

        nombre.substring(espacio + 1)
    };
}

private int contarActoresEnZona(Zona zona) {
    int cuenta = 0;
    for (DelegadoComercial d : estado.getDelegados()) {
        if (zona.equals(d.getZonaActual())) cuenta++;
    }
    for (PatrullaFederal p : estado.getPatrullas()) {
        if (zona.equals(p.getZonaActual())) cuenta++;
    }
    for (Saqueador s : estado.getSaqueadores()) {
        if (zona.equals(s.getZonaActual())) cuenta++;
    }
    return cuenta;
}
}

```

## *PanelRecursos.java*

```

package GUI;

import Concurrencia.SincronizacionRecursos;
import Modelo.DepositoOrbital;
import Modelo.EstadoGalaxia;
import Modelo.TipoRecurso;

import javax.swing.*;
import javax.swing.border.TitledBorder;
import java.awt.*;

/**
 * Muestra el nivel de los tres depositos con barras de progreso y valores numericos
 *
 * El color de cada barra indica el nivel de llenado:
 * - Verde (mas de 60%): nivel saludable
 * - Naranja (30-59%): nivel bajo
 * - Rojo (menos de 30%): nivel critico
 */
public class PanelRecursos extends JPanel {

    private static final Color COLOR_ALTO = new Color(50, 180, 80);

```

```

private static final Color COLOR_MEDIO = new Color(220, 150, 30);
private static final Color COLOR_BAJO = new Color(210, 50, 50);

//Cristal
private JProgressBar barCristal;
private JLabel lblCristalVal;
private JLabel lblCristalPct;

//Mineral
private JProgressBar barMineral;
private JLabel lblMineralVal;
private JLabel lblMineralPct;

//Plasma
private JProgressBar barPlasma;
private JLabel lblPlasmaVal;
private JLabel lblPlasmaPct;

private final SincronizacionRecursos sincRecursos;
private final EstadoGalaxia estado;

public PanelRecursos(SincronizacionRecursos sincRecursos) {
    this.sincRecursos = sincRecursos;
    this.estado = EstadoGalaxia.getInstancia();

    setLayout(new BorderLayout());
    setBorder(new TitledBorder("Depósitos Orbitales"));
    add(crearTablaRecursos(), BorderLayout.CENTER);
}

// =====
// CONSTRUCCION
// =====

private JPanel crearTablaRecursos() {
    JPanel panel = new JPanel(new GridBagLayout());
    panel.setBorder(BorderFactory.createEmptyBorder(6, 8, 6, 8));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 6, 5, 6);
    gbc.fill = GridBagConstraints.HORIZONTAL;
    gbc.anchor = GridBagConstraints.WEST;

    // --- Cabecera ---
    gbc.gridy = 0;

```

```

agregarCabecera(panel, gbc);

// --- Cristal ---
barCristal = crearBarra(new Color(100, 200, 230));
lblCristalVal = new JLabel("0 / 0");
lblCristalPct = new JLabel("0%");
gbc.gridy = 1;
agregarFilaRecurso(panel, gbc, "Cristal", barCristal, lblCristalVal, lblCristalPct);

// --- Mineral ---
barMineral = crearBarra(new Color(180, 140, 80));
lblMineralVal = new JLabel("0 / 0");
lblMineralPct = new JLabel("0%");
gbc.gridy = 2;
agregarFilaRecurso(panel, gbc, "Mineral", barMineral, lblMineralVal, lblMineralPct);

// --- Plasma ---
barPlasma = crearBarra(new Color(220, 100, 180));
lblPlasmaVal = new JLabel("0 / 0");
lblPlasmaPct = new JLabel("0%");
gbc.gridy = 3;
agregarFilaRecurso(panel, gbc, "Plasma", barPlasma, lblPlasmaVal, lblPlasmaPct);

return panel;
}

private void agregarCabecera(JPanel panel, GridBagConstraints gbc) {
    Font fuenteCab = new Font(Font.SANS_SERIF, Font.BOLD, 11);
    String[] cabeceras = {"Recurso", "Nivel", "Unidades", "%"};
    int[] anchos = {80, 220, 90, 50};

    for (int i = 0; i < cabeceras.length; i++) {
        JLabel lbl = new JLabel(cabeceras[i]);
        lbl.setFont(fuenteCab);
        lbl.setForeground(Color.GRAY);
        gbc.gridx = i;
        gbc.weightx = (i == 1) ? 1.0 : 0.0;
        gbc.ipadx = anchos[i];
        panel.add(lbl, gbc);
    }
    gbc.ipadx = 0;
}

```

```

private void agregarFilaRecurso(JPanel panel, GridBagConstraints gbc, String nombre, JProgressBar barra, JLabel lblVal,
JLabel lblPct) {
    gbc.gridx = 0; gbc.weightx = 0;
    panel.add(new JLabel(nombre), gbc);

    gbc.gridx = 1; gbc.weightx = 1.0;
    panel.add(barra, gbc);

    gbc.gridx = 2; gbc.weightx = 0;
    panel.add(lblVal, gbc);

    gbc.gridx = 3;
    panel.add(lblPct, gbc);
}

// =====
// ACTUALIZACION
// =====

//Refresca los valores de los depósitos consultando SincronizacionRecursos
public void refrescar() {
    DepositoOrbital depCristal = estado.getDepositoCristal();
    DepositoOrbital depMineral = estado.getDepositoMineral();
    DepositoOrbital depPlasma = estado.getDepositoPlasma();

    actualizarFila(barCristal, lblCristalVal, lblCristalPct, sincRecursos.getUnidadesActuales(TipoRecurso.CRISTAL),
depCristal.getCapacidadMaximaUnidades());

    actualizarFila(barMineral, lblMineralVal, lblMineralPct, sincRecursos.getUnidadesActuales(TipoRecurso.MINERAL),
depMineral.getCapacidadMaximaUnidades());

    actualizarFila(barPlasma, lblPlasmaVal, lblPlasmaPct, sincRecursos.getUnidadesActuales(TipoRecurso.PLASMA),
depPlasma.getCapacidadMaximaUnidades());
}

// =====
// AUXILIARES
// =====

private void actualizarFila(JProgressBar barra, JLabel lblVal, JLabel lblPct, int actual, int maximo) {
    if (maximo <= 0) return;

    barra.setMaximum(maximo);
    barra.setValue(actual);
}

```

```

double pct = (actual * 100.0) / maximo;
lblVal.setText(actual + " / " + maximo);
lblPct.setText(String.format("%.0f%%", pct));

// Color dinamico segun nivel de llenado
if (pct >= 60) {
    barra.setForeground(COLOR_ALTO);
} else if (pct >= 30) {
    barra.setForeground(COLOR_MEDIO);
} else {
    barra.setForeground(COLOR_BAJO);
}
}

private JProgressBar crearBarra(Color color) {
    JProgressBar barra = new JProgressBar(0, 100);
    barra.setValue(0);
    barra.setStringPainted(false);
    barra.setForeground(color);
    barra.setPreferredSize(new Dimension(200, 20));
    return barra;
}
}

```

### *VentanaPrincipalServidor.java*

```

package GUI;

import Concurrencia.ControlSimulacion;
import Concurrencia.SincronizacionRecursos;
import Log.LoggerFederacionImpl;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

//Ventana principal de la simulación del servidor
public class VentanaPrincipalServidor extends JFrame {

    private static final int INTERVALO_REFRESCO_MS = 500;

```

```

// Paneles
private PanelMapaGalaxia mapa;
private PanelRecursos recursos;
private PanelActores actores;
private PanelLog log;

// Control
private final ControlSimulacion controlSimulacion;
private final SincronizacionRecursos sincRecursos;
private final LoggerFederacionImpl logger;

// Botones
private JButton btnPausar;
private JButton btnReanudar;
private JLabel lblEstado;

// Timer de refresco
private Timer timerRefresco;

public VentanaPrincipalServidor(ControlSimulacion controlSimulacion, SincronizacionRecursos sincRecursos,
LoggerFederacionImpl logger) {
    super("Federación Galáctica de Recursos — Servidor");
    this.controlSimulacion = controlSimulacion;
    this.sincRecursos = sincRecursos;
    this.logger = logger;

    inicializarComponentes();
    inicializarTimer();
    configurarVentana();
}

// =====
// INICIALIZACION
// =====

private void inicializarComponentes() {
    setLayout(new BorderLayout(4, 4));

    // --- Barra superior de control ---
    add(crearPanelControl(), BorderLayout.NORTH);

    // --- Panel central: mapa + pestañas ---
    mapa = new PanelMapaGalaxia();

```

```

recursos = new PanelRecursos(sincRecursos);
actores = new PanelActores();
log = new PanelLog();

JTabbedPane pestanas = new JTabbedPane();
pestanas.addTab("📁 Recursos", recursos);
pestanas.addTab("👥 Actores", actores);
pestanas.addTab("📝 Log", log);
pestanas.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 12));

JSplitPane split = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, mapa, pestanas);
split.setDividerLocation(0.55); //55% de espacio para el mapa
split.setResizeWeight(0.55); //al redimensionar pestaña, crece en la misma proporcion
split.setDividerSize(6);
split.setBorder(new EmptyBorder(4, 4, 4, 4));

setSize(1280, 780);
setMinimumSize(new Dimension(900, 620));

add(split, BorderLayout.CENTER);

// --- Barra de estado inferior ---
add(crearBarraEstado(), BorderLayout.SOUTH);
}

private JPanel crearPanelControl() {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 6));
    panel.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0, Color.LIGHT_GRAY));

    //Titulo
    JLabel titulo = new JLabel("🌌 Federación Galáctica de Recursos");
    titulo.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 14));
    panel.add(titulo);

    //Separador
    panel.add(Box.createHorizontalStrut(20));

    //Boton pausar
    btnPausar = new JButton("⏸ Pausar");
    btnPausar.setBackground(new Color(220, 160, 40));
    btnPausar.setForeground(Color.WHITE);
    btnPausar.setFocusPainted(false);
    btnPausar.setPreferredSize(new Dimension(120, 30));
    btnPausar.addActionListener(e -> pausarSimulacion());

```

```

panel.add(btnPausar);

//Boton reanudar
btnReanudar = new JButton("► Reanudar");
btnReanudar.setBackground(new Color(40, 160, 80));
btnReanudar.setForeground(Color.WHITE);
btnReanudar.setFocusPainted(false);
btnReanudar.setPreferredSize(new Dimension(120, 30));
btnReanudar.setEnabled(false);
btnReanudar.addActionListener(e -> reanudarSimulacion());
panel.add(btnReanudar);

//Etiqueta de estado
lblEstado = new JLabel("● EN MARCHA");
lblEstado.setForeground(new Color(0, 150, 60));
lblEstado.setFont(new Font(Font.SANS_SERIF, Font.BOLD, 12));
panel.add(Box.createHorizontalStrut(16));
panel.add(lblEstado);

return panel;
}

private JPanel crearBarraEstado() {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 3));
    panel.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, Color.LIGHT_GRAY));
    JLabel lbl = new JLabel("Refresco cada " + INTERVALO_REFRESCO_MS + " ms |" + " Log →
federaciongalactica.txt");
    lbl.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 10));
    lbl.setForeground(Color.GRAY);
    panel.add(lbl);
    return panel;
}

// =====
// TIMER DE REFRESCO
// =====

private void inicializarTimer() {
    timerRefresco = new Timer(INTERVALO_REFRESCO_MS, e -> refrescarPaneles());
    timerRefresco.setCoalesce(true); // descarta ticks acumulados
    timerRefresco.start();
}

//Refresca todos los paneles con los datos mas recientes

```



```

private void refrescarPaneles() {
    mapa.refrescar();
    recursos.refrescar();
    actores.refrescar();
}

// =====
// CONTROL DE SIMULACION
// =====

private void pausarSimulacion() {
    controlSimulacion.pausar();
    btnPausar.setEnabled(false);
    btnReanudar.setEnabled(true);
    lblEstado.setText("● PAUSADA");
    lblEstado.setForeground(new Color(200, 130, 0));
    logger.registrarEvento("[GUI] Simulación pausada por el usuario.");
}

private void reanudarSimulacion() {
    controlSimulacion.reanudar();
    btnPausar.setEnabled(true);
    btnReanudar.setEnabled(false);
    lblEstado.setText("● EN MARCHA");
    lblEstado.setForeground(new Color(0, 150, 60));
    logger.registrarEvento("[GUI] Simulación reanudada por el usuario.");
}

// =====
// METODO PUBLICO PARA EL LOG
// =====

//Añade una linea al PanelLog visual
public void agregarLineaLog(String linea) {
    log.agregarLinea(linea);
}

// =====
// CONFIGURACION
// =====

private void configurarVentana() {
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
}

```

```

addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        timerRefresco.stop();
        controlSimulacion.detener();
        dispose();
        System.exit(0);
    }
});
setSize(1024, 680);
setMinimumSize(new Dimension(800, 560));
setLocationRelativeTo(null);
setResizable(true);
}
}

```

## ***Paquete InterfazCliente***

### *ClienteRemotoControlador.java*

```

package InterfazCliente;

import Remoto.EstadoRecursosDTO;
import Remoto.EstadoZonasDTO;
import Remoto.InterfazMonitorRemoto;

import java.rmi.Naming;

/**
 * Hilo controlador del cliente remoto
 * Se conecta al servidor RMI y consulta el estado de la simulacion cada 2 segundos, actualizando la GUI con esos nuevos
 * datos
 * Si la conexcion falla, reintenta en el siguiente ciclo sin detener la aplicacion
 */
public class ClienteRemotoControlador extends Thread {

    private static final int INTERVALO_MS = 2000;
    private static final String URL_RMI = "rmi://127.0.0.1/MonitorGalaxia";

    private InterfazMonitorRemoto monitor;
    private VentanaClienteRemoto ventana;

```

```

private boolean activo;

public ClienteRemotoControlador(VentanaClienteRemoto ventana) {
    this.ventana = ventana;
    this.monitor = null;
    this.activo = true;
    setName("ClienteRemotoController");
    setDaemon(true);
}

@Override
public void run() {
    //Intentar conexion inicial
    conectar();

    while (activo) {
        try {
            Thread.sleep(INTERVALO_MS);

            if (monitor == null) {
                //Reintentar conexion si aun no hay monitor
                conectar();
                continue;
            }

            //Consultar estado al servidor
            EstadoRecursosDTO recursos = monitor.getEstadoRecursos();
            EstadoZonasDTO zonas = monitor.getEstadoZonas();

            // Actualizar GUI en el hilo de eventos
            javax.swing.SwingUtilities.invokeLater(() -> ventana.actualizarEstado(recursos, zonas)
            );

        } catch (java.rmi.RemoteException e) {
            //Conexion perdida: resetear monitor y reintentar
            monitor = null;
            javax.swing.SwingUtilities.invokeLater(() -> ventana.mostrarErrorConexion("Conexión perdida. Reintentando...")
            );
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
}

```

```

//Intenta localizar el objeto remoto en el registro RMI. Si falla, deja monitor a null para reintentar en el siguiente ciclo
private void conectar() {
    try {
        monitor = (InterfazMonitorRemoto) Naming.lookup(URL_RMI);
        javax.swing.SwingUtilities.invokeLater() -> ventana.mostrarMensajeConexion("Conectado al servidor en " +
URL_RMI)
    );
    } catch (Exception e) {
        monitor = null;
        javax.swing.SwingUtilities.invokeLater() -> ventana.mostrarErrorConexion("No se pudo conectar: " +
e.getMessage()
    );
    }
}

//Envia la orden de pausar al servidor remoto
public void pausarRemoto() {
    if (monitor == null) return;
    try {
        monitor.pausarSimulacion();
    } catch (java.rmi.RemoteException e) {
        ventana.mostrarErrorConexion("Error al pausar: " + e.getMessage());
    }
}

//Envia la orden de reanudar
public void reanudarRemoto() {
    if (monitor == null) return;
    try {
        monitor.reanudarSimulacion();
    } catch (java.rmi.RemoteException e) {
        ventana.mostrarErrorConexion("Error al reanudar: " + e.getMessage());
    }
}

//Detiene el hilo controlador limpiamente
public void detener() {
    activo = false;
    interrupt();
}
}

```

## PanelEstadoRemoto.java

```
package InterfazCliente;

import Remoto.EstadoRecursosDTO;
import Remoto.EstadoZonasDTO;

import javax.swing.*;
import javax.swing.border.TitledBorder;
import java.awt.*;

/**
 * Panel del cliente remoto
 * Muestra el estado de los depositos orbitales con barras de progreso y los contadores de actores activos en el sistema
 */
public class PanelEstadoRemoto extends JPanel {

    // --- Depositos: barras de progreso y etiquetas de texto ---
    private JProgressBar barCristal;
    private JProgressBar barMineral;
    private JProgressBar barPlasma;
    private JLabel lblCristalVal;
    private JLabel lblMineralVal;
    private JLabel lblPlasmaVal;

    // --- Contadores de actores ---
    private JLabel lblDelegados;
    private JLabel lblDelegadosActivos;
    private JLabel lblPatrullas;
    private JLabel lblSaqueadores;

    // --- Estado simulación ---
    private JLabel lblEstadoSim;

    public PanelEstadoRemoto() {
        setLayout(new BorderLayout(10, 10));
        setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        add(crearPanelDepositos(), BorderLayout.NORTH);
        add(crearPanelActores(), BorderLayout.CENTER);
    }

    // =====
    // CONSTRUCCION DE SUBPANELES
```

```
// =====
```

```
private JPanel crearPanelDepositos() {
    JPanel panel = new JPanel(new GridBagLayout());
    panel.setBorder(new TitledBorder("Depósitos Orbitales"));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(4, 8, 4, 8);
    gbc.fill = GridBagConstraints.HORIZONTAL;
    gbc.anchor = GridBagConstraints.WEST;

    // --- Fila cabecera ---
    gbc.gridy = 0; gbc.gridx = 0;
    panel.add(new JLabel("Recurso"), gbc);
    gbc.gridx = 1;
    panel.add(new JLabel("Nivel"), gbc);
    gbc.gridx = 2;
    panel.add(new JLabel("Unidades"), gbc);

    // --- Cristal ---
    barCristal = crearBarra(250, new Color(100, 200, 230));
    lblCristalVal = new JLabel("0 / 250");
    gbc.gridy = 1; gbc.gridx = 0;
    panel.add(new JLabel("Cristal"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
    panel.add(barCristal, gbc);
    gbc.gridx = 2; gbc.weightx = 0;
    panel.add(lblCristalVal, gbc);

    // --- Mineral ---
    barMineral = crearBarra(200, new Color(180, 140, 80));
    lblMineralVal = new JLabel("0 / 200");
    gbc.gridy = 2; gbc.gridx = 0;
    panel.add(new JLabel("Mineral"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
    panel.add(barMineral, gbc);
    gbc.gridx = 2; gbc.weightx = 0;
    panel.add(lblMineralVal, gbc);

    // --- Plasma ---
    barPlasma = crearBarra(150, new Color(220, 100, 180));
    lblPlasmaVal = new JLabel("0 / 150");
    gbc.gridy = 3; gbc.gridx = 0;
    panel.add(new JLabel("Plasma"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
```

```

panel.add(barPlasma, gbc);
gbc.gridx = 2; gbc.weightx = 0;
panel.add(lblPlasmaVal, gbc);

return panel;
}

private JPanel crearPanelActores() {
    JPanel panel = new JPanel(new GridBagLayout());
    panel.setBorder(new TitledBorder("Estado del Sistema"));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(6, 12, 6, 12);
    gbc.anchor = GridBagConstraints.WEST;

    //Delegados totales
    gbc.gridy = 0; gbc.gridx = 0;
    panel.add(new JLabel("Delegados totales:"), gbc);
    lblDelegados = new JLabel("—");
    lblDelegados.setFont(lblDelegados.getFont().deriveFont(Font.BOLD));
    gbc.gridx = 1;
    panel.add(lblDelegados, gbc);

    //Delegados activos
    gbc.gridy = 1; gbc.gridx = 0;
    panel.add(new JLabel("Delegados activos:"), gbc);
    lblDelegadosActivos = new JLabel("—");
    lblDelegadosActivos.setFont(lblDelegadosActivos.getFont().deriveFont(Font.BOLD));
    gbc.gridx = 1;
    panel.add(lblDelegadosActivos, gbc);

    //Patrullas
    gbc.gridy = 2; gbc.gridx = 0;
    panel.add(new JLabel("Patrullas federales:"), gbc);
    lblPatrullas = new JLabel("—");
    lblPatrullas.setFont(lblPatrullas.getFont().deriveFont(Font.BOLD));
    gbc.gridx = 1;
    panel.add(lblPatrullas, gbc);

    //Saqueadores
    gbc.gridy = 3; gbc.gridx = 0;
    panel.add(new JLabel("Saqueadores activos:"), gbc);
    lblSaqueadores = new JLabel("—");
    lblSaqueadores.setFont(lblSaqueadores.getFont().deriveFont(Font.BOLD, 13f));
    lblSaqueadores.setForeground(new Color(200, 50, 50));
}

```

```

gbc.gridx = 1;
panel.add(lblSaqueadores, gbc);

//Estado simulacion
gbc.gridy = 4; gbc.gridx = 0;
panel.add(new JLabel("Estado simulación:"), gbc);
lblEstadoSim = new JLabel("Desconocido");
lblEstadoSim.setFont(lblEstadoSim.getFont().deriveFont(Font.BOLD));
gbc.gridx = 1;
panel.add(lblEstadoSim, gbc);

return panel;
}

// =====
// ACTUALIZACION DE DATOS
// =====

//Actualiza todos los componentes con los datos recibidos del servidor RMI
public void actualizarDatos(EstadoRecursosDTO recursos, EstadoZonasDTO zonas) {
    //Depositos
    barCristal.setMaximum(recursos.getCapacidadMaxCristal());
    barCristal.setValue(recursos.getUnidadesCristal());
    lblCristalVal.setText(recursos.getUnidadesCristal() + " / " + recursos.getCapacidadMaxCristal());

    barMineral.setMaximum(recursos.getCapacidadMaxMineral());
    barMineral.setValue(recursos.getUnidadesMineral());
    lblMineralVal.setText(recursos.getUnidadesMineral() + " / " + recursos.getCapacidadMaxMineral());

    barPlasma.setMaximum(recursos.getCapacidadMaxPlasma());
    barPlasma.setValue(recursos.getUnidadesPlasma());
    lblPlasmaVal.setText(recursos.getUnidadesPlasma() + " / " + recursos.getCapacidadMaxPlasma());

    //Actores
    lblDelegados.setText(String.valueOf(zonas.getNumeroDelegados()));
    lblDelegadosActivos.setText(String.valueOf(zonas.getNumeroDelegadosActivos()));
    lblPatrullas.setText(String.valueOf(zonas.getNumeroPatrullas()));
    lblSaqueadores.setText(String.valueOf(zonas.getNumeroSaqueadores()));

    // Estado simulación
    if (zonas.isSimulacionPausada()) {
        lblEstadoSim.setText("PAUSADA");
        lblEstadoSim.setForeground(new Color(200, 130, 0));
    } else {

```



```

        lblEstadoSim.setText("EN MARCHA");
        lblEstadoSim.setForeground(new Color(0, 150, 60));
    }
}

// =====
// AUXILIARES
// =====

//Crea una barra de progreso
private JProgressBar crearBarra(int maximo, Color color) {
    JProgressBar barra = new JProgressBar(0, maximo);
    barra.setValue(0);
    barra.setStringPainted(true);
    barra.setForeground(color);
    barra.setPreferredSize(new Dimension(200, 22));
    return barra;
}
}

```

## *VentanaClienteRemoto.java*

```

package InterfazCliente;

import Remoto.EstadoRecursosDTO;
import Remoto.EstadoZonasDTO;

import javax.swing.*;
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

/**
 * Ventana principal del cliente remoto
 *
 * Contiene:
 * - PanelEstadoRemoto: estado de depositos y actores en tiempo real
 * - Botones: pausar y reanudar la simulacion
 * - Barra de estado: mensajes de conexion y errores RMI
 *
 * El controlador actualiza esta ventana
 */
public class VentanaClienteRemoto extends JFrame {

```

```

private PanelEstadoRemoto panelEstado;
private ClienteRemotoControlador controlador;

//Botones
private JButton btnPausar;
private JButton btnReanudar;

//Barra de estado inferior
private JLabel lblBarraEstado;

public VentanaClienteRemoto() {
    super("Monitor Remoto – Federación Galáctica de Recursos");
    inicializarComponentes();
    inicializarControlador();
    configurarVentana();
}

// =====
// INICIALIZACION
// =====

private void inicializarComponentes() {
    setLayout(new BorderLayout(8, 8));

    // Panel principal de estado
    panelEstado = new PanelEstadoRemoto();
    add(panelEstado, BorderLayout.CENTER);

    // Panel de botones
    add(crearPanelBotones(), BorderLayout.SOUTH);

    // Barra de estado
    lblBarraEstado = new JLabel(" Conectando al servidor...");
    lblBarraEstado.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createMatteBorder(1, 0, 0, 0,
    Color.LIGHT_GRAY), BorderFactory.createEmptyBorder(3, 8, 3, 8))
    );
    lblBarraEstado.setFont(lblBarraEstado.getFont().deriveFont(11f));
    lblBarraEstado.setForeground(Color.DARK_GRAY);
    add(lblBarraEstado, BorderLayout.NORTH);
}

private JPanel crearPanelBotones() {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 16, 8));

```

```

panel.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, Color.LIGHT_GRAY));

btnPausar = new JButton("⏸ Pausar simulacion");
btnPausar.setBackground(new Color(220, 160, 40));
btnPausar.setForeground(Color.WHITE);
btnPausar.setFocusPainted(false);
btnPausar.setPreferredSize(new Dimension(200, 36));
btnPausar.addActionListener(e -> controlador.pausarRemoto());

btnReanudar = new JButton("▶ Reanudar simulación");
btnReanudar.setBackground(new Color(40, 160, 80));
btnReanudar.setForeground(Color.WHITE);
btnReanudar.setFocusPainted(false);
btnReanudar.setPreferredSize(new Dimension(200, 36));
btnReanudar.addActionListener(e -> controlador.reanudarRemoto());

panel.add(btnPausar);
panel.add(btnReanudar);
return panel;
}

private void inicializarControlador() {
    controlador = new ClienteRemotoControlador(this);
    controlador.start();
}

private void configurarVentana() {
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            controlador.detener();
            dispose();
        }
    });
    setSize(480, 380);
    setMinimumSize(new Dimension(420, 340));
    setLocationRelativeTo(null);
    setResizable(true);
}

// =====
// METODOS PUBLICOS (llamados desde ClienteRemotoControlador)
// =====

```

```

//Actualiza todos los paneles con los datos recibidos del servidor
public void actualizarEstado(EstadoRecursosDTO recursos, EstadoZonasDTO zonas) {
    panelEstado.actualizarDatos(recursos, zonas);

    // Sincronizar estado de botones con el estado real
    btnPausar.setEnabled(!zonas.isSimulacionPausada());
    btnReanudar.setEnabled(zonas.isSimulacionPausada());
}

//Muestra mensaje informativo en la barra de estado
public void mostrarMensajeConexion(String mensaje) {
    lblBarraEstado.setText(" ✓ " + mensaje);
    lblBarraEstado.setForeground(new Color(0, 130, 60));
}

//Muestra mensaje de error en la barra de estado
public void mostrarErrorConexion(String mensaje) {
    lblBarraEstado.setText(" ✖ " + mensaje);
    lblBarraEstado.setForeground(new Color(190, 40, 40));
    btnPausar.setEnabled(false);
    btnReanudar.setEnabled(false);
}
}

```

## ***Paquete Log***

### *EventoLog.java*

```

package Log;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Representa un evento del sistema
 * Encapsula el timestamp, el actor implicado y la descripción del evento para ser escrito en el fichero log
 *
 * Formato de salida: "2026-06-14 12:35:04 Descripción del evento."
 */
public class EventoLog {

```

```

private static final DateTimeFormatter FORMATO = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

private LocalDateTime timestamp;
private String descripcion;

public EventoLog(String descripcion) {
    this.timestamp = LocalDateTime.now();
    this.descripcion = descripcion;
}

// --- Getters ---

public LocalDateTime getTimestamp() {
    return timestamp;
}

public String getDescripcion() {
    return descripcion;
}

//Devuelve la línea completa lista para escribir en el fichero
public String getLineaFormateada() {
    return timestamp.format(FORMATO) + " " + descripcion;
}

@Override
public String toString() {
    return getLineaFormateada();
}
}

```

### *LoggerFederacion.java*

```

package Log;

//Define el contrato minimo que debe cumplir cualquier implementacion del logger del sistema

```

```
public interface LoggerFederacion {

    //Registra un evento en el log
    void registrarEvento(String descripcion);
}
```

### LoggerFederacionImpl.java

```
package Log;

import GUI.VentanaPrincipalServidor;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import javax.swing.SwingUtilities;

/**
 * Implementacion thread-safe del logger, protegido por ReentrantLock explicito
 * El fichero se abre en modo append para no borrar el historial
 */
public class LoggerFederacionImpl implements LoggerFederacion {

    private static final String NOMBRE_FICHERO = "federacion_galactica.txt";

    private final Lock lock;

    public LoggerFederacionImpl() {
        this.lock = new ReentrantLock();
    }

    private VentanaPrincipalServidor ventana;

    public void setVentana(VentanaPrincipalServidor ventana) {
        this.ventana = ventana;
    }

    @Override
    public void registrarEvento(String descripcion) {
        EventoLog evento = new EventoLog(descripcion);
        String lineaFormateada = evento.getLineaFormateada();
        lock.lock();
```

```

    try {
        escribirEnFichero(evento.getLineaFormateada());
    } finally {
        lock.unlock();
    }

    if (ventana != null) {
        SwingUtilities.invokeLater(() -> ventana.agregarLineaLog(lineaFormateada));
    }
}

//Abre y cierra el fichero por cada registro para garantizar que los eventos queden registrados incluso si el programa se
cierra de forma inesperada
private void escribirEnFichero(String linea) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(NOMBRE_FICHERO, true))) {
        writer.write(linea);
        writer.newLine();
    } catch (IOException e) {
        System.err.println("Error al escribir en el log: " + e.getMessage());
    }
}
}

```

## ***Paquete Main***

### *MainClienteRemoto.java*

```

package Main;

import InterfazCliente.VentanaClienteRemoto;

import javax.swing.*;

//Arranca la ventana del cliente
public class MainClienteRemoto {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            VentanaClienteRemoto ventana =
                new VentanaClienteRemoto();

```

```

        ventana.setVisible(true);
    });
}
}

```

## MainServidor.java

```

package Main;

import Concurrencia.*;
import GUI.VentanaPrincipalServidor;
import Log.LoggerFederacionImpl;
import Modelo.EstadoGalaxia;
import Remoto.ServidorRMI;

import javax.swing.*;

public class MainServidor {

    public static void main(String[] args) {

        // 1. Logger
        LoggerFederacionImpl logger = new LoggerFederacionImpl();

        // 2. Sincronizacion
        SincronizacionZonas sincZonas = new SincronizacionZonas();
        SincronizacionRecursos sincRecursos = new SincronizacionRecursos();
        SincronizacionSaqueadores sincSaq = new SincronizacionSaqueadores();

        // 3. Control de simulacion
        ControlSimulacion control = new ControlSimulacion();

        // 4. Servidor RMI
        ServidorRMI servidorRMI = new ServidorRMI(control, sincRecursos);
        servidorRMI.iniciar();

        // 5. GUI
        SwingUtilities.invokeLater(() -> {
            VentanaPrincipalServidor ventana = new VentanaPrincipalServidor(control, sincRecursos, logger);

            logger.setVentana(ventana); //enlace GUI-logger
            ventana.setVisible(true);
        });
    }
}

```



```

        control.iniciar();

        logger.registrarEvento("[Main] Simulación iniciada.");

        // 6. Generadores de actores
        GeneradorDelegados generadorDel = new GeneradorDelegados(sincZonas, sincRecursos, control, logger);

        GeneradorPatrullas generadorPat = new GeneradorPatrullas(sincZonas, sincSaq, control, logger);

        GeneradorSaqueadores generadorSaq = new GeneradorSaqueadores(sincZonas, sincRecursos, sincSaq, control,
        logger);

        generadorDel.start();
        generadorPat.start();
        generadorSaq.start();

        logger.registrarEvento("[Main] Simulación iniciada.");
    });
}
}

```

## ***Paquete Modelo***

### **Actor.java**

```

package Modelo;

/**
 * Representa a cualquier actor del sistema (delegado, patrulla o saqueador)
 * Todos los actores tienen un id unico y pertenecen a una zona en cada momento
 */
public abstract class Actor {

    private String id;
    private Zona zonaActual;
    private String estado;
    private boolean enRecuperacion;

    public Actor(String id) {
        this.id = id;
        this.zonaActual = null;
    }
}

```

```

        this.estado = "Iniciando";
        this.enRecuperacion = false;
    }

    // --- Getters y Setters ---

    public String getId() {
        return id;
    }

    public Zona getZonaActual() {
        return zonaActual;
    }

    public void setZonaActual(Zona zonaActual) {
        this.zonaActual = zonaActual;
    }

    public String getEstado() {
        return estado;
    }

    public void setEstado(String estado) {
        this.estado = estado;
    }

    public boolean isEnRecuperacion() {
        return enRecuperacion;
    }

    public void setEnRecuperacion(boolean enRecuperacion) {
        this.enRecuperacion = enRecuperacion;
    }

    @Override
    public String toString() {
        return id + " [" + estado + "]" + (zonaActual != null ? " en " + zonaActual.getNombre() : "");
    }
}

```

### *BaseSaqueadores.java*

```
package Modelo;
```

```

/**
 * Esperan tras aparecer (3-6s) y tras completar un ataque (10s de espera obligatoria, o 20s si fueron derrotados)
 * Limite de 40 saqueadores
 */
public class BaseSaqueadores extends Zona {

    public BaseSaqueadores() {
        super("Base de Saqueadores", Integer.MAX_VALUE);
    }
}

```

### *CentroCoordinacionFederal.java*

```

package Modelo;

/**
 * Zona central donde los delegados comerciales esperan entre ciclos de trabajo, preparando su siguiente solicitud de recursos
 * Sin limite de ocupacion
 */
public class CentroCoordinacionFederal extends Zona {

    public CentroCoordinacionFederal() {
        super("Centro de Coordinación Federal", Integer.MAX_VALUE);
    }
}

```

### *DelegadoComercial.java*

```

package Modelo;

/**
 * Extraen recursos de los planetas mineros y los depositan en los depositos orbitales
 * Se identifican como DXXX (ej: D001)
 * Su ciclo de trabajo lo ejecuta HiloDelegado en Concurrencia
 */
public class DelegadoComercial extends Actor {

    private TipoRecurso recursoObjetivo;
    private int unidadesTransportadas;
    private boolean expulsado;

    public DelegadoComercial(String id) {

```

```

    super(id);
    this.recursoObjetivo = null;
    this.unidadesTransportadas = 0;
    this.expulsado = false;
}

// --- Getters y Setters ---

public TipoRecurso getRecursoObjetivo() {
    return recursoObjetivo;
}

public void setRecursoObjetivo(TipoRecurso recursoObjetivo) {
    this.recursoObjetivo = recursoObjetivo;
}

public int getUnidadesTransportadas() {
    return unidadesTransportadas;
}

public void setUnidadesTransportadas(int unidadesTransportadas) {
    this.unidadesTransportadas = unidadesTransportadas;
}

public boolean isExpulsado() {
    return expulsado;
}

public void setExpulsado(boolean expulsado) {
    this.expulsado = expulsado;
}
}

```

### *DepositoOrbital.java*

```

package Modelo;

```

```

/**

```

\* Deposito donde los delegados almacenan los recursos extraidos de los planetas

\*

\* Tiene dos limites diferenciados:

\* Acceso: max 3 delegados depositando a la vez

\* Capacidad: max de unidades que puede almacenar

\*

\* Depositos del sistema:

\* Cristal -> capacidad 250 u.

\* Mineral -> capacidad 200 u.

\* Plasma -> capacidad 150 u.

\*/

```
public class DepositoOrbital extends Zona {
```

```
    private static final int MAX_DELEGADOS_DEPOSITO = 3;
```

```
    private TipoRecurso tipoRecurso;
```

```
    private int capacidadMaximaUnidades;
```

```
    private int unidadesActuales;
```

```
    public DepositoOrbital(String nombre, TipoRecurso tipoRecurso, int capacidadMaximaUnidades, int unidadesIniciales) {
```

```
        super(nombre, MAX_DELEGADOS_DEPOSITO);
```

```
        this.tipoRecurso = tipoRecurso;
```

```
        this.capacidadMaximaUnidades = capacidadMaximaUnidades;
```

```
        this.unidadesActuales = unidadesIniciales;
```

```
    }
```

```
    // --- Getters y Setters ---
```

```
    public TipoRecurso getTipoRecurso() {
```

```
        return tipoRecurso;
```

```
    }
```

```
    public int getCapacidadMaximaUnidades() {
```

```
        return capacidadMaximaUnidades;
```

```
    }
```

```
    public int getUnidadesActuales() {
```

```
        return unidadesActuales;
```

```
    }
```

```
    public void setUnidadesActuales(int unidadesActuales) {
```

```
        this.unidadesActuales = unidadesActuales;
```

```
    }
```

```
// --- Metodos de consulta de estado ---

//Devuelve el espacio libre disponible en unidades
public int getEspacioDisponible() {
    return capacidadMaximaUnidades - unidadesActuales;
}

//Indica si hay espacio suficiente para almacenar la cantidad dada
public boolean tieneEspacio(int cantidad) {
    return (unidadesActuales + cantidad) <= capacidadMaximaUnidades;
}

//Indica si el deposito esta por debajo del umbral de alerta de desabastecimiento (10% de la capacidad max)
//Usado por el generador de delegados para decidir si crear uno nuevo
public boolean estaPorDebajoUmbral() {
    return unidadesActuales < (capacidadMaximaUnidades * 0.10);
}
}
```

## *EstadoGalaxia.java*

```
package Modelo;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Contenedor global del estado de la simulacion galactica
 * Contiene:
 * - Zonas del sistema (planetas, depositos, base, hangar, etc.)
 * - Listas de actores activos (delegados, patrullas, saqueadores)
 * - Generadores de IDs unicos
 */
public class EstadoGalaxia {

    // --- Instancia Singleton ---

    private static EstadoGalaxia instancia;
```

```

public static synchronized EstadoGalaxia getInstancia() {
    if (instancia == null) {
        instancia = new EstadoGalaxia();
    }
    return instancia;
}

// =====
// ZONAS
// =====

private final CentroCoordinacionFederal centroCoordinacion;
private final HangarPatrullas hangarPatrullas;
private final BaseSaqueadores baseSaqueadores;
private final ZonaRecuperacion zonaRecuperacion;

// Planetas mineros
private final PlanetaMinero planetaCryon;
private final PlanetaMinero planetaVelora;
private final PlanetaMinero planetaFerrum;
private final PlanetaMinero planetaDrax;
private final PlanetaMinero planetaIgnis;

// Lista unificada de planetas
private final List<PlanetaMinero> planetas;

// Depositos orbitales
private final DepositoOrbital depositoCristal;
private final DepositoOrbital depositoMineral;
private final DepositoOrbital depositoPlasma;

// Lista unificada de depositos
private final List<DepositoOrbital> depositos;

// =====
// ACTORES ACTIVOS
// =====

private final List<DelegadoComercial> delegados;
private final List<PatrullaFederal> patrullas;
private final List<Saqueador> saqueadores;

// =====
// GENERADORES DE IDs UNICOS (thread-safe con AtomicInteger)

```

```
// =====

private final AtomicInteger contadorDelegados;
private final AtomicInteger contadorPatrullas;
private final AtomicInteger contadorSaqueadores;

// =====
// CONSTRUCTOR PRIVADO
// =====

private EstadoGalaxia() {

    // --- Zonas generales ---
    centroCoordinacion = new CentroCoordinacionFederal();
    hangarPatrullas = new HangarPatrullas();
    baseSaqueadores = new BaseSaqueadores();
    zonaRecuperacion = new ZonaRecuperacion();

    // --- Planetas mineros ---
    planetaCryon = new PlanetaMinero("Cryon", TipoRecurso.CRISTAL);
    planetaVelora = new PlanetaMinero("Velora", TipoRecurso.CRISTAL);
    planetaFerrum = new PlanetaMinero("Ferrum", TipoRecurso.MINERAL);
    planetaDrax = new PlanetaMinero("Drax", TipoRecurso.MINERAL);
    planetalgnis = new PlanetaMinero("Ignis", TipoRecurso.PLASMA);

    planetas = new ArrayList<>();
    planetas.add(planetaCryon);
    planetas.add(planetaVelora);
    planetas.add(planetaFerrum);
    planetas.add(planetaDrax);
    planetas.add(planetalgnis);

    // --- Depositos orbitales (nombre, recurso, capacidad max., unidades iniciales) ---
    depositoCristal = new DepositoOrbital("Depósito de Cristal", TipoRecurso.CRISTAL, 250, 100);
    depositoMineral = new DepositoOrbital("Depósito de Mineral", TipoRecurso.MINERAL, 200, 80);
    depositoPlasma = new DepositoOrbital("Depósito de Plasma", TipoRecurso.PLASMA, 150, 60);

    depositos = new ArrayList<>();
    depositos.add(depositoCristal);
    depositos.add(depositoMineral);
    depositos.add(depositoPlasma);

    // --- Listas de actores ---
    delegados = new ArrayList<>();

```



```

    patrullas = new ArrayList<>();
    saqueadores = new ArrayList<>();

    // --- Contadores de IDs ---
    contadorDelegados = new AtomicInteger(0);
    contadorPatrullas = new AtomicInteger(0);
    contadorSaqueadores = new AtomicInteger(0);
}

// =====
// GENERADORES DE IDs
// =====

//Genera el siguiente ID unico para delegado
public String generarIdDelegado() {
    return String.format("D%03d", contadorDelegados.incrementAndGet());
}

//Genera el siguiente ID unico para patrulla
public String generarIdPatrulla() {
    return String.format("P%03d", contadorPatrullas.incrementAndGet());
}

//Genera el siguiente ID único para saqueador
public String generarIdSaqueador() {
    return String.format("S%03d", contadorSaqueadores.incrementAndGet());
}

// =====
// GETTERS DE ZONAS
// =====

public CentroCoordinacionFederal getCentroCoordinacion() {
    return centroCoordinacion;
}

public HangarPatrullas getHangarPatrullas() {
    return hangarPatrullas;
}

public BaseSaqueadores getBaseSaqueadores() {
    return baseSaqueadores;
}

```

```

public ZonaRecuperacion getZonaRecuperacion() {
    return zonaRecuperacion;
}

public PlanetaMinero getPlanetaCryon() {return planetaCryon;}
public PlanetaMinero getPlanetaVelora() {return planetaVelora;}
public PlanetaMinero getPlanetaFerrum() {return planetaFerrum;}
public PlanetaMinero getPlanetaDrax() {return planetaDrax;}
public PlanetaMinero getPlanetaIgnis() {return planetaIgnis;}

public List<PlanetaMinero> getPlanetas() {
    return planetas;
}

public DepositoOrbital getDepositoCristal() {return depositoCristal;}
public DepositoOrbital getDepositoMineral() {return depositoMineral;}
public DepositoOrbital getDepositoPlasma() {return depositoPlasma;}

public List<DepositoOrbital> getDepositos() {
    return depositos;
}

//Devuelve todas las zonas del sistema en orden fijo para el mapa
public List<Zona> getTodasLasZonas() {
    List<Zona> zonas = new ArrayList<>();
    zonas.add(centroCoordinacion);
    zonas.addAll(planetas); // lista de 3 planetas
    zonas.addAll(depositos); // lista de 3 depósitos
    zonas.add(hangarPatrullas);
    zonas.add(baseSaqueadores);
    zonas.add(zonaRecuperacion);
    return Collections.unmodifiableList(zonas);
}

// =====
// GETTERS Y METODOS DE ACTORES
// =====

public synchronized void agregarDelegado(DelegadoComercial delegado) {
    delegados.add(delegado);
}

public synchronized void eliminarDelegado(DelegadoComercial delegado) {
    delegados.remove(delegado);
}

```

```

}

public synchronized List<DelegadoComercial> getDelegados() {
    return new ArrayList<>(delegados);
}

public synchronized int getNumeroDelegados() {
    return delegados.size();
}

public synchronized int getNumeroDelegadosActivos() {
    int activos = 0;
    for (DelegadoComercial d : delegados) {
        if (!d.isEnRecuperacion()) {
            activos++;
        }
    }
    return activos;
}

public synchronized void agregarPatrulla(PatrullaFederal patrulla) {
    patrullas.add(patrulla);
}

public synchronized void eliminarPatrulla(PatrullaFederal patrulla) {
    patrullas.remove(patrulla);
}

public synchronized List<PatrullaFederal> getPatrullas() {
    return new ArrayList<>(patrullas);
}

public synchronized int getNumeroPatrullas() {
    return patrullas.size();
}

public synchronized void agregarSaqueador(Saqueador saqueador) {
    saqueadores.add(saqueador);
}

public synchronized void eliminarSaqueador(Saqueador saqueador) {
    saqueadores.remove(saqueador);
}

```

```

    public synchronized List<Saqueador> getSaqueadores() {
        return new ArrayList<>(saqueadores);
    }

    public synchronized int getNumeroSaqueadores() {
        return saqueadores.size();
    }
}

```

### HangarPatrullas.java

```

package Modelo;

/**
 * Zona donde las patrullas federales inician su actividad,
 * preparando sus sistemas entre 3 y 6 segundos, y a donde
 * regresan tras ser derrotadas en combate
 * Sin limite de ocupacion
 */
public class HangarPatrullas extends Zona {

    public HangarPatrullas() {
        super("Hangar de Patrullas", Integer.MAX_VALUE);
    }
}

```

### PatrullaFederal.java

```

package Modelo;

/**
 * Las patrullas defienden planetas y depositos frente a los
 * saqueadores, patrullando aleatoriamente entre zonas
 * Se identifican como PXXX (ej: P001)
 * Su ciclo de trabajo lo ejecuta HiloPatrulla en Concurrencia.
 */
public class PatrullaFederal extends Actor {

    private boolean enCombate;
    private int combatesGanados;
    private int combatesPerdidos;

    public PatrullaFederal(String id) {

```

```

    super(id);
    this.enCombate = false;
    this.combatesGanados = 0;
    this.combatesPerdidos = 0;
}

// --- Getters y Setters ---

public boolean isEnCombate() {
    return enCombate;
}

public void setEnCombate(boolean enCombate) {
    this.enCombate = enCombate;
}

public int getCombatesGanados() {
    return combatesGanados;
}

public void setCombatesGanados(int combatesGanados) {
    this.combatesGanados = combatesGanados;
}

public int getCombatesPerdidos() {
    return combatesPerdidos;
}

public void setCombatesPerdidos(int combatesPerdidos) {
    this.combatesPerdidos = combatesPerdidos;
}

//Incrementa contador de victorias
public void registrarVictoria() {
    this.combatesGanados++;
}

//Incrementa contador de derrotas
public void registrarDerrota() {
    this.combatesPerdidos++;
}
}

```

## PlanetaMinero.java

```
package Modelo;

/**
 * Cada planeta produce un recurso distinto y permite
 * maximo 4 delegados extrayendo a la vez
 * Los planetas disponen de recursos ilimitados.
 *
 * Planetas del sistema:
 * Cryon -> CRISTAL
 * Velora -> CRISTAL
 * Ferrum -> MINERAL
 * Drax -> MINERAL
 * Ignis -> PLASMA
 */
public class PlanetaMinero extends Zona {

    private static final int MAX_DELEGADOS_PLANETA = 4;

    private TipoRecurso tipoRecurso;

    public PlanetaMinero(String nombre, TipoRecurso tipoRecurso) {
        super(nombre, MAX_DELEGADOS_PLANETA);
        this.tipoRecurso = tipoRecurso;
    }

    // --- Getters ---

    public TipoRecurso getTipoRecurso() {
        return tipoRecurso;
    }
}
```

## Saqueador.java

```
package Modelo;

/**
 * Ataca planetas mineros y depositos orbitales
```

- \*
  - \* Se identifican como SXXX
  - \* Su ciclo de trabajo lo ejecuta HiloSaqueador en Concurrencia
  - \*
    - \* Probabilidad de objetivo:
      - \* 70% -> deposito
      - \* 30% -> planeta minero
    - \* Probabilidad de victoria en combate: 50% para cada bando

```

*/
public class Saqueador extends Actor {

    //Fases del ciclo de vida de un saqueador
    public enum FaseAtaque {
        EN_BASE,
        DESPLAZANDOSE,
        EN_COMBATE,
        SAQUEANDO,
        REGRESANDO
    }

    private Zona objetivo;
    private FaseAtaque faseActual;
    private int unidadesRobadas;

    public Saqueador(String id) {
        super(id);
        this.objetivo = null;
        this.faseActual = FaseAtaque.EN_BASE;
        this.unidadesRobadas = 0;
    }

    // --- Getters y Setters ---

    public Zona getObjetivo() {
        return objetivo;
    }

    public void setObjetivo(Zona objetivo) {
        this.objetivo = objetivo;
    }

    public FaseAtaque getFaseActual() {
        return faseActual;
    }

```

```

    }

    public void setFaseActual(FaseAtaque faseActual) {
        this.faseActual = faseActual;
    }

    public int getUnidadesRobadas() {
        return unidadesRobadas;
    }

    public void setUnidadesRobadas(int unidadesRobadas) {
        this.unidadesRobadas = unidadesRobadas;
    }

    //Indica si el saqueador esta atacando alguna zona (en combate o en fase de saqueo activo)
    public boolean estaAtacando() {
        return faseActual == FaseAtaque.EN_COMBATE || faseActual == FaseAtaque.SAQUEANDO;
    }
}

```

### *TipoRecurso.java*

```

package Modelo;

//Tipos de recursos en la galaxia
public enum TipoRecurso {
    CRISTAL,
    MINERAL,
    PLASMA
}

```

### *Zona.java*

```

package Modelo;

/**
 * Representa una zona del sistema galáctico.
 * Las zonas tienen nombre, capacidad máxima de actores

```



```

* y estado de ataque.
*/
public abstract class Zona {

    private String nombre;
    private int capacidadMaxima;
    private int ocupacionActual;
    private boolean bajoAtaque;

    public Zona(String nombre, int capacidadMaxima) {
        this.nombre = nombre;
        this.capacidadMaxima = capacidadMaxima;
        this.ocupacionActual = 0;
        this.bajoAtaque = false;
    }

    // --- Getters y Setters ---

    public String getNombre() {
        return nombre;
    }

    public int getCapacidadMaxima() {
        return capacidadMaxima;
    }

    public int getOcupacionActual() {
        return ocupacionActual;
    }

    public void setOcupacionActual(int ocupacionActual) {
        this.ocupacionActual = ocupacionActual;
    }

    public boolean isBajoAtaque() {
        return bajoAtaque;
    }

    public void setBajoAtaque(boolean bajoAtaque) {
        this.bajoAtaque = bajoAtaque;
    }

    public boolean estaLlena() {
        return ocupacionActual >= capacidadMaxima;
    }
}

```

```

    }

    @Override
    public String toString() {
        return nombre;
    }
}

```

### *ZonaRecuperacion.java*

```

package Modelo;

/**
 * Zona a la que son enviados los delegados expulsados durante un ataque y las patrullas derrotadas en combate
 * Sin limite de ocupacion
 */
public class ZonaRecuperacion extends Zona {

    public ZonaRecuperacion() {
        super("Zona de Recuperación", Integer.MAX_VALUE);
    }
}

```

## ***Paquete Remoto***

### *EstadoRecursosDTO.java*

```

package Remoto;

import java.io.Serializable;

/**
 * DTO (Data Transfer Object) serializable con el estado actual de los tres depositos
 * Se envia desde el servidor al cliente remoto
 */
public class EstadoRecursosDTO implements Serializable {

    private static final long serialVersionUID = 1L;

    private final int unidadesCristal;
    private final int capacidadMaxCristal;
}

```

```

private final int unidadesMineral;
private final int capacidadMaxMineral;
private final int unidadesPlasma;
private final int capacidadMaxPlasma;

public EstadoRecursosDTO(int unidadesCristal, int capacidadMaxCristal, int unidadesMineral, int capacidadMaxMineral,
int unidadesPlasma, int capacidadMaxPlasma) {
    this.unidadesCristal = unidadesCristal;
    this.capacidadMaxCristal = capacidadMaxCristal;
    this.unidadesMineral = unidadesMineral;
    this.capacidadMaxMineral = capacidadMaxMineral;
    this.unidadesPlasma = unidadesPlasma;
    this.capacidadMaxPlasma = capacidadMaxPlasma;
}

public int getUnidadesCristal() {return unidadesCristal;}
public int getCapacidadMaxCristal() {return capacidadMaxCristal;}
public int getUnidadesMineral() {return unidadesMineral;}
public int getCapacidadMaxMineral() {return capacidadMaxMineral;}
public int getUnidadesPlasma() {return unidadesPlasma;}
public int getCapacidadMaxPlasma() {return capacidadMaxPlasma;}
}

```

### EstadoZonasDTO.java

```

package Remoto;

import java.io.Serializable;

//DTO serializable con contadores de actores activos agrupados por tipo, para visualizacion en cliente remoto
public class EstadoZonasDTO implements Serializable {

    private static final long serialVersionUID = 1L;

    private final int numeroDelegados;
    private final int numeroDelegadosActivos;
    private final int numeroPatrullas;
    private final int numeroSaqueadores;
    private final boolean simulacionPausada;

    public EstadoZonasDTO(int numeroDelegados, int numeroDelegadosActivos, int numeroPatrullas, int
numeroSaqueadores, boolean simulacionPausada) {
        this.numeroDelegados = numeroDelegados;

```

```

        this.numeroDelegadosActivos = numeroDelegadosActivos;
        this.numeroPatrullas = numeroPatrullas;
        this.numeroSaqueadores = numeroSaqueadores;
        this.simulacionPausada = simulacionPausada;
    }

    public int getNumeroDelegados() {return numeroDelegados;}
    public int getNumeroDelegadosActivos() {return numeroDelegadosActivos;}
    public int getNumeroPatrullas() {return numeroPatrullas;}
    public int getNumeroSaqueadores() {return numeroSaqueadores;}
    public boolean isSimulacionPausada() {return simulacionPausada;}
}

```

### *InterfazMonitorRemoto.java*

```

package Remoto;

import java.rmi.Remote;
import java.rmi.RemoteException;

/**
 * Interfaz remota de la Federacion Galactica
 * Define los servicios que el servidor ofrece al cliente remoto:
 * - Consulta del estado de recursos (depositos)
 * - Consulta del estado de actores por zona
 * - Control remoto de la simulacion (pausa/reanudación)
 */
public interface InterfazMonitorRemoto extends Remote {

    //Devuelve el estado actual de los tres depositos
    EstadoRecursosDTO getEstadoRecursos() throws RemoteException;

    //Devuelve los contadores de actores en el sistema
    EstadoZonasDTO getEstadoZonas() throws RemoteException;

    //Pausa la simulacion
    void pausarSimulacion() throws RemoteException;

    //Reanuda la simulacion pausada
    void reanudarSimulacion() throws RemoteException;

    //Indica si la simulacion esta pausada en el momento
    boolean isPausada() throws RemoteException;
}

```

```
}
```

### MonitorRemotoImpl.java

```
package Remoto;

import Concurrencia.ControlSimulacion;
import Concurrencia.SincronizacionRecursos;
import Modelo.DepositoOrbital;
import Modelo.EstadoGalaxia;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

/**
 * Implementacion del monitor remoto
 * Delega todas las consultas en EstadoGalaxia y ControlSimulacion, que ya son thread-safe, por lo que no necesita
 * sincronizacion extra
 */
public class MonitorRemotoImpl extends UnicastRemoteObject implements InterfazMonitorRemoto {

    private static final long serialVersionUID = 1L;

    private final EstadoGalaxia estado;
    private final ControlSimulacion control;
    private final SincronizacionRecursos sincRecursos;

    public MonitorRemotoImpl(ControlSimulacion control, SincronizacionRecursos sincRecursos) throws RemoteException {
        super(); // Registra el objeto como exportable
        this.estado = EstadoGalaxia.getInstancia();
        this.control = control;
        this.sincRecursos = sincRecursos;
    }

    @Override
    public EstadoRecursosDTO getEstadoRecursos() throws RemoteException {
        DepositoOrbital cristal = estado.getDepositoCristal();
        DepositoOrbital mineral = estado.getDepositoMineral();
        DepositoOrbital plasma = estado.getDepositoPlasma();

        return new EstadoRecursosDTO(
            sincRecursos.getUnidadesActuales(Modelo.TipoRecurso.CRISTAL),
            cristal.getCapacidadMaximaUnidades(),

```

```

        sincRecursos.getUnidadesActuales(Modelo.TipoRecurso.MINERAL),
        mineral.getCapacidadMaximaUnidades(),
        sincRecursos.getUnidadesActuales(Modelo.TipoRecurso.PLASMA),
        plasma.getCapacidadMaximaUnidades()
    );
}

@Override
public EstadoZonasDTO getEstadoZonas() throws RemoteException {
    return new EstadoZonasDTO(
        estado.getNumeroDelegados(),
        estado.getNumeroDelegadosActivos(),
        estado.getNumeroPatrullas(),
        estado.getNumeroSaqueadores(),
        control.isPausada()
    );
}

@Override
public void pausarSimulacion() throws RemoteException {
    control.pausar();
}

@Override
public void reanudarSimulacion() throws RemoteException {
    control.reanudar();
}

@Override
public boolean isPausada() throws RemoteException {
    return control.isPausada();
}
}

```

## *ServidorRMI.java*

```

package Remoto;

import Concurrencia.ControlSimulacion;
import Concurrencia.SincronizacionRecursos;

import java.rmi.Naming;

```

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

//Arranca el registro RMI y publica el objeto MonitorRemotoImpl para ser consultado
public class ServidorRMI {

    private static final int PUERTO_RMI = 1099;
    private static final String NOMBRE_RMI = "MonitorGalaxia";

    private final ControlSimulacion control;
    private final SincronizacionRecursos sincRecursos;

    private Registry registry;

    public ServidorRMI(ControlSimulacion control, SincronizacionRecursos sincRecursos) {
        this.control = control;
        this.sincRecursos = sincRecursos;
        this.registry = null;
    }

    //Crea el registro local y publica el objeto remoto, debe llamarse desde MainServidor antes de iniciar la simulacion
    public void iniciar() {
        try {
            //Crear registro local en puerto
            registry = LocateRegistry.createRegistry(PUERTO_RMI);

            //Crear e instanciar el objeto remoto
            MonitorRemotoImpl monitor = new MonitorRemotoImpl(control, sincRecursos);

            //Publicar el objeto en el registro con su nombre
            Naming.rebind("rmi://127.0.0.1/" + NOMBRE_RMI, monitor);

            System.out.println("ServidorRMI: objeto " + NOMBRE_RMI + " registrado en puerto " + PUERTO_RMI + ".");

        } catch (Exception e) {
            System.err.println("ServidorRMI: error al iniciar RMI: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```